# AI-POWERED HEALTHCARE MANAGEMENT SYSTEM

A Role-Based Web Application for Clinic Appointment Booking, AI-Assisted Healthcare Support, Prescription Management, and Patient Records

A Project Report Submitted in Fulfillment of the Requirements for Vocational Training

SUBMITTED BY:
Majji Lokesh Krishna

UNDER THE GUIDANCE OF:
Dr. Archana Sharma

# CERTIFICATE

This is to certify that the project report entitled "AI-Powered Healthcare Management System" represents the original work successfully completed by **Majji Lokesh Krishna**, University Roll Number: 300111324021, a student of **Bhilai Institute of Technology, Durg**, Semester **IV**, Year **2025–2026**. This project has been submitted to Techonet as part of the requirements for Vocational Training and stands as an authentic record of the work carried out under supervision.

_____

Project Supervisor:
**Dr. Archana Sharma**

# ACKNOWLEDGEMENTS

The successful completion of this project, HealthCare Portal, would not have been possible without the guidance, support, and encouragement of several people.

I would like to express my sincere gratitude to my project supervisor for the constant guidance, constructive feedback, and patience shown throughout the design, development, and documentation of this project. Their input was especially valuable during the decisions around system architecture, database design, and authentication, discussed in later chapters of this report.

I am also thankful to the faculty of the Department of Computer Science & Engineering for providing access to the resources, references, and technical environment needed to complete this project within the given timeline.

Finally, I would like to thank my family and friends for their continued support and encouragement during the course of this project.

**Majji Lokesh Krishna**

# TABLE OF CONTENTS



# LIST OF FIGURES



# LIST OF TABLES

# CHAPTER 1: INTRODUCTION

## 1.1 Project Overview

Most small clinics still run on paper. Appointments are written into a register, prescriptions are handwritten, and a patient's past visits are scattered across whatever files happen to be on hand that day. Large hospitals solved this problem years ago with dedicated hospital information systems, but that kind of software is usually too expensive and too complex for a small clinic to adopt. The HealthCare Portal was built to close that gap, at least for the core, everyday parts of running a clinic.

It is a web application with three types of users: Patient, Doctor, and Receptionist. Each one logs into the same system but sees a different dashboard depending on their role. A patient can book an appointment, upload a lab report, and check their prescriptions. A doctor can see their schedule, write a prescription, and add notes to a patient's file. A receptionist manages the appointment list for the whole clinic and approves or reschedules bookings as needed. None of these roles share a dashboard — the interface a doctor sees after logging in looks nothing like what a patient sees, even though both are hitting the same backend.

On the technical side, the frontend is built with plain HTML, CSS, and JavaScript, so there is no heavy framework to set up or maintain. The backend is an Express.js server running on Node.js, and it talks to a MongoDB database using Mongoose. Login is handled with JWT, and passwords are hashed with bcrypt before they are stored, so the actual password is never kept anywhere in the database. File uploads — lab reports, scans, and so on — are handled using Multer, which saves the file to disk and keeps a record in the database of which patient it belongs to and what type of document it is.

In addition to the core healthcare management features, the system integrates a cloud-based Generative AI module to improve both doctor and patient experiences. Doctors can generate AI-powered summaries of a patient's medical history by analysing previous prescriptions, consultation notes, and uploaded medical documents, enabling faster review before

consultations. Patients can also interact with an AI assistant that explains medical terminology, summarizes medical reports in simple language, and answers basic healthcare-related questions. These AI features are designed to support healthcare workflows while ensuring that all clinical decisions remain the responsibility of qualified healthcare professionals.

None of these technologies were picked because they are the newest or most impressive option available. They were picked because they are well documented, free to use, and reasonably simple to work with for a project of this size. A production hospital system would likely need much more — audit logging, redundancy, formal compliance checks — but for a clinic-scale tool built as a project, this stack was enough to get a working, secure system without spending most of the time fighting the tools themselves.

To make the overall idea concrete, it helps to walk through what actually happens during a single visit. A patient logs in and books a slot with a doctor of their choice. That booking shows up immediately on the receptionist's dashboard as a pending request, and the receptionist can approve it, move it to a different time, or reject it if the slot is no longer available. Once approved, it also appears on the doctor's schedule for that day. When the patient arrives and the consultation happens, the doctor updates the appointment, writes a prescription if needed, and can add a note to the patient's file. The patient can then log back in later and see exactly what was prescribed, without having to remember it or keep track of a paper slip.

## 1.2 Background

Clinic record-keeping has changed a lot over the last few decades, though not evenly across the board. For a long time, everything was paper based — appointment books, prescription pads, and folders full of test results. This worked, in the sense that clinics ran on it for decades, but it had obvious problems. A folder could go missing. Two patients could accidentally be booked into the same slot if two different staff members were writing into the register at the same time. A doctor seeing a returning patient had no quick way to check what happened during the last visit unless the old file was physically retrieved and happened to be complete.

Bigger hospitals started adopting Electronic Health Record systems from around the 1990s onward, but this shift mostly stayed limited to large institutions that could afford dedicated IT

teams and expensive licensing. Small clinics were largely left out of this wave, and in a lot of places, they still are. It was only with the growth of cloud hosting and cheaper web infrastructure through the 2010s that lighter, web-based clinic management tools started becoming realistic for smaller practices to consider. Patient-facing appointment booking, in particular, became far more common during this period, and it picked up even more after the pandemic pushed a lot of clinics toward remote or low-contact booking simply out of necessity rather than preference.

Even with all of this, there is still a noticeable gap between what a large hospital runs on and what a typical neighborhood clinic actually uses. A lot of small practices either stay entirely on paper or patch together a mix of tools — a spreadsheet here, a messaging app there — that were never really designed to work together. That gap is the space this project is aimed at. It is not trying to compete with an enterprise hospital system; it is trying to give a small clinic something better than a paper register and a stack of loose files, without asking them to take on the cost or complexity of enterprise software.

The HealthCare Portal fits into this second, lighter category of tools. It is not trying to be a full hospital information system, and it does not attempt to cover billing, insurance claims, or inventory management the way a large HIS would. It is trying to solve the specific, everyday problems that come up in a small clinic: booking appointments, writing prescriptions, and keeping a patient's history in one place instead of several.

## 1.3 Motivation

The idea for this project came from noticing how much friction still exists in a normal clinic visit, even a fairly routine one. Booking an appointment usually means calling the clinic during office hours and hoping the receptionist can find an open slot without double-booking someone else. If the line is busy or the clinic is closed, the patient has no way to check availability on their own — there is no way for them to see the schedule themselves, so they are stuck waiting or calling back later.

On the doctor's side, the bigger issue is history. A patient who has visited the clinic three or four times before should ideally have a doctor who can quickly check what was diagnosed or

prescribed during those earlier visits, especially if the reason for the current visit is related. In practice, that almost never happens unless the same doctor happened to see them each time and personally remembers, or unless someone manually pulls out the old paper file in time for the appointment. If the clinic has more than one doctor, this problem gets worse, since a second doctor may have no visibility into what the first one did at all.

There is also the problem of lost documents. Patients are handed lab reports and scans on paper, and these get misplaced between visits fairly often — sometimes because the patient loses them, sometimes because the clinic itself does. A doctor then either has to work without that information or ask for the test to be repeated, which wastes time and, in most cases, costs the patient money for a test that was already done once. These three problems — scheduling, history tracking, and document loss — are the main things this project set out to fix, and each one maps fairly directly onto a feature in the final system: real-time appointment status, linked patient history, and document upload.

It is also worth mentioning that none of these problems are unique to any one clinic. They come up in more or less the same form across small practices in general, which is part of why building a reusable, general-purpose system made more sense than building something tied to one specific clinic's exact workflow.

### 1.3.1 A Common Scenario

A concrete example makes the motivation easier to see. Consider a patient who visited the clinic two months ago for a persistent cough, was prescribed antibiotics, and is now coming back because the symptoms returned. In a paper-based clinic, the receptionist would need to physically locate that patient's old file, assuming it wasn't misfiled or handed to someone else in the meantime. If a different doctor is on duty that day, they would be starting from scratch, relying entirely on whatever the patient remembers to mention about the earlier visit.

With the portal, the same scenario looks different. The patient books a slot through the app. Whichever doctor picks up the appointment can open the patient's record and immediately see the earlier prescription, the dosage that was given, and any notes from that visit. If the earlier treatment didn't work, that's visible right away instead of being reconstructed from a patient's

memory of what pills they were given two months ago. This is a fairly small scenario, but it's representative of the kind of everyday friction the system was built to remove.

## 1.4 Objectives of the Project

The objectives below were kept deliberately practical, since the goal was to build something that actually works end to end rather than something that tries to cover every possible feature a real hospital system might eventually need.

- Build a single web platform connecting Patients, Doctors, and Receptionists, with each role getting its own dashboard rather than a shared, one-size-fits-all interface.
- Implement secure login and role-based access using JWT, so that a patient cannot accidentally, or deliberately, access doctor-only or receptionist-only data.
- Let patients book, view, and track their own appointments without needing to call the clinic or rely on a staff member being available.
- Allow doctors to record diagnoses, write prescriptions, and add notes tied to a specific patient and appointment, so the record stays organized by visit.
- Give receptionists a working view of the day's appointments so they can approve, reschedule, or cancel bookings as needed, without having to coordinate over phone or in person.
- Support secure upload of medical documents, such as lab reports and scans, linked to the correct patient rather than stored as loose, unlabeled files.
- Reduce how much of the clinic's day-to-day work still depends on paper and manual coordination between staff.

In addition to the core clinic management workflow, the system integrates a Generative AI module that enhances both doctor and patient interactions. Doctors can generate concise summaries of a patient's medical history, prescriptions, and uploaded reports before consultation, while patients can use an AI assistant to understand medical terminology and receive simplified explanations of their reports. These AI features are designed to improve accessibility and efficiency while supporting, rather than replacing, professional medical judgment.

## 1.5 Scope of the Project

The system is built around three roles and five main data types: Users, Appointments, Prescriptions, Notes, and Uploads. Within that, it covers registration and login, appointment

booking with status tracking, writing prescriptions, adding clinical notes, and uploading documents. Every feature in the system maps back to one of these five collections in some way, which kept the data model reasonably easy to reason about even as more features were added during development.

A few things were left out on purpose, mostly to keep the project achievable within the time available rather than because they weren't considered. There is no online payment integration yet, though the appointment schema already has a fee amount and a feePaid flag, so this can be added later without changing the database structure. There is no video consultation feature, no automatic SMS or email reminders, and no separate mobile app — the system is web-only for now, accessed through a browser on any device.

These were not left out by accident. They were cut from this version specifically to keep the project manageable, and they are covered again in the Future Scope chapter as things that could reasonably be added on top of the current system without needing to redesign it from scratch. The underlying data model was, in fact, designed with a few of these additions in mind, which is why fields like feePaid already exist even though the payment flow itself is not built yet.

## 1.6 Target Users

It's worth being specific about who this system is actually built for, since that shapes a lot of the smaller design decisions. The primary target is a small to mid-sized clinic — somewhere between a single-doctor practice and a facility with maybe five or six doctors sharing a receptionist. A large multi-department hospital is not really the intended audience; at that scale, the assumptions the system makes (one shared appointment list, one central receptionist role) start to break down.

Within that clinic, the three roles map to three different comfort levels with technology, and the interface for each was kept as simple as possible with that in mind. A receptionist may be juggling phone calls and walk-ins at the same time as using the dashboard, so their view avoids anything that isn't directly related to managing the schedule. A doctor is usually short on time between patients, so the prescription and notes forms are kept quick to fill in rather than asking

for excessive detail. A patient, who may be using the system for the first time under some amount of stress, gets the simplest view of all — book, check status, view results.

## 1.7 System Features

The features are grouped by role, since each dashboard is built around what that particular type of user actually needs to do, and nothing more. Keeping each dashboard narrow was a deliberate choice — a receptionist does not need to see clinical notes, and a patient does not need to see the full day's schedule for every doctor.

### 1.7.1 Patient

- Register and log in using an email and password, with the password stored as a bcrypt hash rather than in plain text.
- Book an appointment with a doctor of their choice and track its status (pending, waiting, in progress, done, or cancelled) as it changes.
- Upload medical documents, tagged by type — lab report, scan, prescription, or other — so that a doctor reviewing the file later knows what they're looking at.
- View past prescriptions and notes written by their doctor, tied to the specific appointment each one came from.

### 1.7.2 Doctor

- View their own appointment schedule, separate from any other doctor's schedule at the same clinic.
- Write a prescription, including the medicine name, dosage, frequency, duration, and any extra instructions.
- Add notes to a patient's file, linked to a specific appointment rather than floating unattached in the patient's record.
- Review any documents the patient has uploaded ahead of or during the consultation.

### 1.7.3 Receptionist

- View the full appointment schedule across all doctors at the clinic, not just one.
- Approve, reschedule, or cancel appointments as needed, updating the status so patients and doctors both see the change.
- Handle walk-in or phone-based booking requests by creating or adjusting appointments directly on behalf of a patient.

## 1.8 How the System Works: A Typical Appointment Lifecycle

It is easier to understand how the pieces fit together by following one appointment from start to finish, rather than just listing features in isolation.

It starts with the patient. After logging in, they pick a doctor and a time slot and submit a booking request. At this point, the appointment is created in the database with a status of pending — nothing has been confirmed yet. This request immediately becomes visible to the receptionist, who can see it alongside every other pending request for the day.

The receptionist then reviews it. If the slot is genuinely free, they approve it, and the status changes to waiting. If there's a conflict, they can suggest a different time instead, effectively rescheduling it before it is ever confirmed. Once approved, the appointment shows up on the relevant doctor's own schedule, filtered so the doctor only sees appointments assigned to them.

On the day of the visit, the doctor moves the appointment to in progress once the consultation begins. During or after the consultation, they can write a prescription — adding one or more medicines, each with its own dosage and duration — and separately add a clinical note if there's anything worth recording beyond the prescription itself, such as an observation for a future visit. Once everything is recorded, the appointment is marked done.

From the patient's side, none of this requires a phone call. They can check the status of their appointment at any point, and once it's marked done, the prescription and any notes the doctor chose to share become visible in their own dashboard. If, at any stage, either the patient or the receptionist needs to cancel instead, the status simply moves to cancelled, and the slot becomes available again for someone else to book.

The upload workflow runs alongside this but isn't strictly tied to a single appointment. A patient can upload a lab report or scan at any time, tag it with a type, and it becomes part of their permanent record, visible to any doctor who treats them going forward — not just the doctor they happen to be seeing that day.

## 1.9 Advantages

A few advantages became clear once the system was actually built and tested, not just in theory.

- All of a patient's appointments, prescriptions, and notes are linked together, so a doctor can pull up the full history in one place instead of piecing it together from memory or paper files.
- Because JWT does not need a session stored on the server, the login system stays simple even if the backend is later run across more than one server instance.
- Prescriptions are stored as structured data — a list of medicines with dosage and duration — rather than free text, which avoids the legibility problems that come with handwriting and makes it possible to look back at exactly what was prescribed.
- Everyone — patient, doctor, and receptionist — can see the same appointment status at the same time, so there is no need for a phone call just to check if a slot is free.
- The whole stack is open source, so there are no licensing costs involved in running it, which matters for a small clinic working with a limited budget.
- Because each role only sees its own dashboard, there's less risk of a receptionist accidentally viewing clinical notes, or a patient seeing another patient's data.

## 1.10 Disadvantages

The system also has real limitations, and it's worth being upfront about them rather than glossing over them.

- It needs an internet connection to work at all — there is no offline mode, so a network outage at the clinic would stop bookings and record access temporarily.
- It has not gone through any formal security audit or compliance review, such as HIPAA or a similar regulation, so it should not be treated as ready for handling real patient data without further work.
- There is no payment processing, video consultation, or automatic notification system yet, so the clinic would still need to handle those parts separately for now.
- Uploading lab reports still has to be done manually — the system does not connect directly to any diagnostic lab to pull results automatically, so there's always a manual step between a test being done and it showing up in the portal.
- Since this was built as a project rather than a commercial product, it has not been tested at the scale a busy, multi-doctor clinic with hundreds of patients a day would need.

## 1.11 Existing System

Clinics without any dedicated software today generally fall into one of two situations. Some still run entirely on paper — a register for appointments, a prescription pad, and a filing cabinet for records. This needs no technical setup, but it is hard to search through, easy to lose, and only one person can look at the register at a time, which becomes a real bottleneck during busy hours.

Others have moved partway to digital tools without really integrating them — a spreadsheet for appointments, a word processor for typing prescriptions, and a folder of scanned documents somewhere on a computer. This is a small step up from pure paper, but there is still no access control between staff, and nothing actually connects an appointment to the prescription or notes that came out of it. If a receptionist and a doctor are both editing the same spreadsheet, there's also a real risk of one person overwriting the other's changes without noticing.

On the other end of the spectrum, large hospital information systems solve most of these problems, but they are usually too expensive and too complicated for a small clinic to justify. They're built for institutions with dozens of departments and hundreds of staff, and a lot of that complexity is simply wasted on a clinic with two or three doctors and a single receptionist. In between these two extremes, there isn't much — which is really the gap this project is aimed at.

## 1.12 Proposed System

The HealthCare Portal sits between these two extremes. It keeps the structure of a proper system — appointments, prescriptions, and notes are all linked to the right patient and doctor — while staying simple enough that a small team could realistically build, deploy, and maintain it without a dedicated IT department.

Each role gets a dashboard suited to what they actually need: patients book their own appointments and see their own history, doctors manage consultations and issue prescriptions, and receptionists keep an eye on the overall schedule. Nothing extra is bolted on beyond what these three roles need to do their part of the workflow, which keeps the interface manageable for staff who may not be especially comfortable with technology to begin with.

The To further enhance the core healthcare workflow, the system integrates a cloud-based Generative AI module. Doctors can generate concise summaries of a patient's medical history by analysing previous prescriptions, consultation notes, and uploaded medical documents, allowing them to review relevant information more efficiently before consultations. Patients can also interact with an AI assistant that explains medical terminology, summarizes medical reports in simple language, and answers basic healthcare-related questions. These AI features are intended to improve efficiency and accessibility while serving only as supportive tools, with all clinical decisions remaining under the responsibility of qualified healthcare professionals.

The goal throughout was to solve the real, everyday problems described in Section 1.3 — scheduling friction, fragmented history, and lost documents — without pulling in the cost and complexity of a full hospital information system. The addition of AI-powered assistance further improves the usability of the system by reducing the time required to review patient records and making healthcare information easier for patients to understand. Whether these objectives were fully met is something the Results and Discussion chapter looks at more closely, based on how the system actually performed during testing.

### 1.12.1 Assumptions and Constraints

A few assumptions were made while designing the system, and it's worth listing them rather than leaving them implicit. The system assumes each clinic runs its own separate deployment — it is not built as a multi-tenant platform where several unrelated clinics share one database. It assumes a doctor belongs to exactly one clinic at a time, and it assumes the receptionist role is trusted with administrative access across all doctors, since that's how reception typically works in a small practice.

On the constraint side, the project was built within a fixed timeframe as part of a vocational or academic submission, which is the main reason features like payments and notifications were scoped out rather than attempted in a rushed or incomplete form. The reasoning throughout was that a smaller set of features working reliably is more useful, and more honestly demonstrable, than a longer feature list where half the pieces are unfinished.

## 1.13 Source Code Repository

The complete source code for this project is available on GitHub at
https://github.com/Lokesh-0916/AI-Powered-Healthcare-Management-System
Detailed setup, installation, and usage instructions are provided in the repository's README file.

## 1.14 Report Organization

The rest of this report is organized as follows. Chapter 2 covers the Literature Review, looking at existing clinic management platforms and the technologies considered for this project. Chapter 3 states the Problem Identification in more concise terms, pulling together the motivation described here into a formal problem statement. Chapter 4 covers the Methodology and System Design, including the architecture, the SDLC model used, the technology stack, the database design, and the UML diagrams. Chapter 5 presents the Results and Discussion, with implementation details, screenshots, and the outcomes of testing. Chapter 6 covers the Future Scope, including the features that were deliberately left out of this version, and Chapter 7 wraps up with the Conclusion.

# CHAPTER 2: LITERATURE REVIEW

## 2.1 Purpose of this Review

Before picking a stack or drawing up the database schema, it made sense to look at what clinics are actually using right now — the well-known platforms, and the manual setups a lot of small clinics still fall back on. This isn't a literature review in the strict academic sense of going through a stack of published papers one by one. It's closer to sizing up real, working tools against the actual problem this project is trying to solve. The question that mattered here wasn't "what does the research say about EHR adoption" but "what can the clinics this is meant for actually get their hands on today, and where does that fall short."

Where a real source exists — a platform's own docs, a government program page, a published case study — it's named and linked at the end of the chapter instead of being dressed up as a numbered citation for something that was never actually read. A lot of student project reports fill their literature review with citations like "Smith et al., 2019" attached to findings that sound plausible but are made up. That wasn't done here, since a fake citation is worse than no citation at all — it just looks rigorous while being the opposite.

The rest of the chapter goes like this. Section 2.2 covers how clinic and hospital software has changed over the last few decades, including the push toward digital health infrastructure in India. Section 2.3 looks at three kinds of existing solutions — manual and spreadsheet setups, consumer platforms like Practo, and open-source systems like OpenMRS. Section 2.4 goes through the specific technology choices made for this project and why each one made more sense than the obvious alternative. Section 2.5 wraps up with a gap analysis that ties back to the objectives from Chapter 1.

## 2.2 Evolution of Health Information Systems

Clinic record-keeping has gone through roughly three overlapping phases, though the timeline has never been even across the industry, and a lot of that unevenness is still visible today.

The first phase — paper registers, prescription pads, and filing cabinets — is the one most small clinics are still partly or fully stuck in, as described in Chapter 1. It has no setup cost and no learning curve, but it does not scale, and it makes any history lookup dependent on someone physically finding the right folder. Two staff writing into the same register at the same time can double-book a slot without either realizing it.

The second phase began in large hospitals from around the 1990s onward, with the adoption of dedicated Electronic Health Record (EHR) and Hospital Information Systems (HIS) — expensive, licensed products built for institutions with dozens of departments and a dedicated IT staff, never designed with a two- or three-doctor clinic's budget in mind.

The third phase, where this project sits, is the rise of lighter, web-based clinic tools aimed at smaller providers. This became realistic through the 2010s as cloud hosting got cheap enough that a small clinic didn't need its own server room, and JavaScript frameworks matured enough for a small team to build a capable booking system without an enterprise budget. Patient-facing booking grew a lot in this period, and picked up further after the pandemic pushed clinics toward remote or low-contact booking.

Open-source medical record platforms grew alongside this third phase. OpenMRS, discussed in Section 2.3.3, was first released in 2004 specifically to give under-resourced clinics an EHR option without a licensing bill attached — a useful marker for how long this exact gap has been recognized without being fully closed for the smallest end of the market.

### 2.2.1 The Indian Context: ABDM and Digital Health Infrastructure

Since this project is realistically aimed at clinics operating in India, it is worth looking at how digital health infrastructure has developed there specifically. The Indian government launched the Ayushman Bharat Digital Mission (ABDM) in September 2021, run by the National Health Authority, aiming to build a secure, consent-based, interoperable digital health ecosystem connecting patients, providers, labs, and pharmacies. Central to ABDM is the Ayushman Bharat Health Account (ABHA) — a unique digital health identifier a patient can use to link records across providers, similar in spirit to how Aadhaar functions as a general identity number.

ABDM matters here for two reasons. First, it shows the direction of national policy is toward digitized, interoperable records, which supports the basic premise of this project even at small-clinic scale. Second, larger commercial platforms such as Practo are increasingly expected to integrate with ABDM's consent framework, adding regulatory and technical complexity that a lightweight standalone tool like the HealthCare Portal does not currently carry. This project does not integrate with ABDM — a deliberate scope decision, listed again in the Future Scope chapter as a realistic later direction.

### 2.2.2 Mobile Access and the Shift Toward Patient Self-Service

A parallel trend worth noting is the shift toward patients managing their own healthcare interactions directly rather than going through clinic staff. A decade ago, checking a doctor's availability meant calling during office hours; today, patients increasingly expect to check availability, book, and view their own history on their own schedule.

This shift shows up differently across the platforms reviewed here. Practo's consumer app is built almost entirely around patient self-service, with Ray acting as the backend those bookings flow into. OpenMRS, by contrast, has historically been more clinician- and administrator-facing, reflecting its origins in public health record-keeping. This project sits closer to the Practo end of that spectrum in intent — patients log in and manage their own appointments and history directly — while staying well short of Practo's broader marketplace ambitions.

### 2.2.3 Where This Leaves Small Clinics Today

Bringing the previous two sections together: national policy is pushing toward interoperable digital records, patients increasingly expect to book online rather than call, and yet the tools available to a small clinic are either a large commercial platform bundled with features it may not need, a powerful but technically demanding open-source EMR, or an enterprise HIS priced for a hospital. That mismatch is the space this project is aimed at, developed further in the platform-by-platform review that follows.

## 2.3 Review of Existing Platforms and Approaches

Three categories of existing approach are directly relevant: the manual and semi-digital setups most small clinics actually use, consumer-facing platforms like Practo, and open-source EMR systems like OpenMRS built for resource-constrained settings. A fourth category, large

enterprise Hospital Information Systems, is included briefly for contrast even though it isn't realistic at this scale.

### 2.3.1 Manual and Spreadsheet-Based Setups

This is not a commercial product, but it is still the most common "system" at small clinics, so it belongs here as the baseline. In its purest form it is a paper register, a prescription pad, and a filing cabinet. A common middle ground is a spreadsheet for the schedule, a word processor for prescriptions, and a folder of scanned reports on a shared computer.

The spreadsheet version is searchable and does not physically get lost, and it is essentially free. But it does not solve the deeper problems: there is no access control, so anyone with the file can see or edit any patient's row, and there is no real link between an appointment, its prescription, and any notes — reconstructing history still means manually cross-referencing files. Because spreadsheets are usually edited by one person at a time, two staff working on the same sheet risk silently overwriting each other's changes.

The relevance of this category is mostly negative — it defines the floor. Any replacement needs, at minimum, role-based access, a single source of truth linking appointments to prescriptions and notes, and safe concurrent access. All three were treated as non-negotiable baseline requirements the HealthCare Portal was built around from the first schema design.

### 2.3.2 Practo Ray

Practo is the platform most Indian patients and clinics would recognize by name, and its practice management product, Ray, is a reasonable stand-in for a modern, well-funded commercial clinic system. Ray handles scheduling, digital prescriptions, and billing, and — through the wider Practo ecosystem — patient discovery. Practo's own materials describe Ray as trusted by over fifty thousand clinic owners, running on HIPAA-compliant infrastructure hosted on AWS, with a stated partnership with India's Data Security Council. Ray's marketing also leans on features aimed at growing a clinic's patient base — listing in Practo's "My Doctors," multilingual support, virtual-number calling — reflecting Practo's broader identity as a patient-facing discovery and booking marketplace, with Ray as the practice-management layer plugged into it.

For a small clinic, Ray's strength is also its limitation: it is a full practice-and-marketing platform bundled together, tied into Practo's own discovery product — useful for a clinic wanting more walk-in discovery, but unnecessary overhead for a clinic that just wants scheduling and records. Being proprietary and hosted also means recurring subscription costs with no option to self-host, and Ray's feature set extends into billing, multi-location support, and marketing analytics well beyond what this project set out to build — a small clinic needing only the core booking-prescription-records loop ends up paying for, and navigating, a considerably larger product than it needs day to day.

### 2.3.3 OpenMRS

OpenMRS sits almost opposite Practo, in both cost and origin. It is a free, open-source EMR platform, first released in 2004 with involvement from the Regenstrief Institute (affiliated with Indiana University) and Partners In Health, alongside a global community working in low-resource settings, and it reportedly runs across roughly eight thousand facilities in more than seventy countries, including deployments supported by Médecins Sans Frontières. As a records system it is genuinely comprehensive — customizable patient data forms without programming, longitudinal history tracking, and integration with public health surveillance for conditions like HIV, tuberculosis, and malaria — and being open source, it has no licensing fee, with REST and FHIR APIs built for integration with other clinical tools.

Where it becomes a harder fit is deployment reality. Its configurability is a genuine strength for large public health programs adapting one core system to many contexts, but customizing it for a specific clinic's workflow typically assumes in-house technical capacity a small clinic usually does not have. Its interface is also oriented around structured clinical data entry and public health reporting rather than the tighter, receptionist-managed appointment lifecycle a walk-in clinic runs on, and its funding base has historically centered on public health and humanitarian deployments rather than the commercial-clinic use case this project targets.

### 2.3.4 Enterprise Hospital Information Systems

At the top end sit large, proprietary Hospital Information Systems built for multi-department hospitals, with modules for billing, pharmacy, radiology, and insurance claims all tied together. Systems such as Epic and Oracle Health (formerly Cerner) are well-known examples,

mentioned here only to make the comparison concrete — their implementation details are generally restricted to licensed institutional customers.

Their pricing, implementation timelines, and staffing requirements are built around institutions with a permanent IT team. Very little of that is relevant, or affordable, for a clinic with two or three doctors and one receptionist — even the parts that would theoretically be useful come bundled with unrelated functionality a small clinic has no use for. Enterprise HIS platforms are included here mainly to frame the far end of the spectrum, with Practo and OpenMRS occupying different positions in between, and the HealthCare Portal aiming at a gap none of the three fully covers.

### 2.3.5 Reading the Four Approaches Together

Read side by side, the four approaches trace a fairly clear line. Manual and spreadsheet setups cost nothing but provide essentially no structure or access control. Enterprise HIS sits at the opposite extreme — extensive capability at a cost that puts it out of reach for a small clinic. Practo Ray and OpenMRS sit in between but arrive from different directions: Practo starts from a commercial booking marketplace and adds practice management underneath; OpenMRS starts from a highly configurable clinical records platform never primarily built around a tight, receptionist-managed booking loop. None of the four is a bad system within its own context — the point is that none was built around the specific combination a small, single-location clinic has: a tight budget, no dedicated IT staff, exactly three roles, and a need for the core loop to just work without much surrounding configuration.

### 2.3.6 A Rough Cost and Effort Comparison

Paper and spreadsheet setups cost nothing upfront, but the ongoing cost shows up as staff time lost to manual cross-referencing and the occasional serious error a digital system would have prevented. Enterprise HIS sits at the other extreme: large upfront cost plus an ongoing requirement for trained IT staff, for functionality a small clinic will mostly never use. Practo Ray falls in the middle, with a recurring subscription but very low effort to start since Practo hosts everything. OpenMRS is free to license, the lowest direct cost of the four after paper, but its effort cost shifts almost entirely onto setup and configuration. The HealthCare Portal was built to sit close to OpenMRS on direct cost while requiring considerably less configuration effort, since its scope was kept narrow from the outset.

### 2.3.7 Data Privacy and Compliance Considerations

One dimension worth reviewing separately is data privacy and regulatory compliance, since the platforms reviewed take noticeably different approaches reflecting the regulatory environment each was built for.

Practo explicitly markets its infrastructure as HIPAA-compliant and cloud-hosted, reflecting both the seriousness a commercial platform has to treat this with and the reality that clinics trust a third party with patient data. OpenMRS, being self-hosted, places compliance responsibility largely on whoever deploys it, while manual paper and spreadsheet systems generally have no formal compliance framework at all — arguably the weakest position despite feeling most familiar to staff. This project sits closest to the OpenMRS end: self-hosted, so a clinic keeps its own data on infrastructure it controls, but it has not gone through any formal compliance audit — HIPAA, India's Digital Personal Data Protection Act, or an equivalent — and Section 1.10 already states this as a disadvantage. Self-hosting is a genuine privacy advantage in one specific sense, but it isn't, by itself, equivalent to formal regulatory compliance, and any clinic considering this for real patient data would need to treat a compliance review as a prerequisite, not an afterthought.

| Platform / Approach | Primary Target Scale | Upfront & Recurring Cost | Technical Setup & Maintenance Effort | Key Drawback / Gap Addressed |
|---|---|---|---|---|
| Paper & Spreadsheets | Single practitioner / small clinic | Free / negligible | None | Lacks role-based access control, searchability, and data linkages. |
| Practo Ray | Commercial clinics seeking discovery | Recurring subscription fee | Low (vendor-hosted cloud) | Bundles unnecessary marketing/billing features; recurring SaaS cost. |
| OpenMRS | Resource-constrained public health facilities | Free & Open Source | High (requires specialized IT customization) | Complex deployment; tailored for public health data rather than a tight clinic booking loop. |
| Enterprise HIS (e.g., Epic) | Multi-department hospitals | High licensing & infrastructure costs | Very High (requires dedicated IT team) | Prohibitively expensive and overly complex for small practices. |
| HealthCare Portal (Proposed) | Small to mid-sized single-location clinics | Free & Open Source (Self-hostable) | Low (lightweight Node.js/MongoDB stack) | Delivers core scheduling, prescriptions, and document upload without enterprise bloat. |

*Table 2.1 — Comparative Evaluation of Clinic Management Approaches*

## 2.4 Technology Choices in Context

Reviewing existing platforms answers what problem needed solving and roughly where the gap sits. This section covers why the specific technologies used to build the HealthCare Portal — Node.js and Express, MongoDB, JWT-based authentication, and Multer for file handling — made sense compared with the more commonly taught alternatives.

### 2.4.1 Node.js and Express vs. Django and Spring

Django (Python) and Spring (Java) are both mature, well-documented frameworks that could realistically have done the job. Node and Express were chosen instead for two practical reasons rather than any claim of general superiority: Node lets the same language — JavaScript — run on both frontend and backend, cutting context-switching for a small project built by one or two people on a fixed timeline, and Express is deliberately minimal, which suited a bounded feature set better than a "batteries included" alternative like Django's built-in admin panel and ORM, or Spring's heavier configuration and dependency-injection conventions, both more suited to a larger team maintaining a long-lived system.

There is also a concrete technical reason tied back to Section 2.3.1's observation about double-booking in manual systems: Node's single-threaded, event-loop concurrency model handles simultaneous requests — for instance, two browser tabs booking the same slot — through a non-blocking queue rather than a new thread per request, and combined with a database-level check before confirming a booking, this makes it straightforward to reject a conflicting booking cleanly. Django and Spring can achieve the same guarantee through their own concurrency models, so this is not a claim that Node is uniquely capable, only that its default model lined up naturally here.

### 2.4.2 MongoDB vs. MySQL

The choice between MongoDB and a relational database came down mostly to how the data is naturally shaped. A prescription is a variable-length list of medicines, each with its own dosage, frequency, and duration. Modeling that in a relational schema means a separate, linked medicines table with a join required every read. In MongoDB, the same prescription is naturally one document with an embedded array of medicines, matching how it is actually used — read and written as one complete unit.

Mongoose, the ODM layer used on top of MongoDB, adds schema definition and validation — required fields, enumerated status values, reference fields — without forcing every nested list into its own table and chain of joins. This isn't a universal argument that document databases suit healthcare data in general — relational databases still have a real advantage for anything depending on strict transactional integrity across linked tables, billing being the obvious example, which is one reason this project's scope deliberately leaves full payment processing for later, even though the appointment schema already includes a feePaid flag in preparation. For the current scope, the flexibility of a document model outweighed the transactional guarantees a relational schema would add, since none of the current features depend on those stricter guarantees yet.

### 2.4.3 JWT vs. Session-Based Authentication

Session-based authentication stores a session identifier on the server and a matching cookie on the client, requiring the server to track every session somewhere. JWT works differently: the server issues a signed token containing the user's identity and role, and every request carries that token instead of relying on session state — the server only verifies the signature. For this project, statelessness was the main draw: authentication doesn't depend on the backend running as a single instance, which matters if the API ever needs to run across more than one server. The trade-off is that a JWT can't easily be revoked before it expires the way a session can simply be deleted — say, the moment a staff member is let go — but that was judged acceptable given a short expiry window and the fact that this is an internal tool rather than a large, constantly churning consumer app.

### 2.4.4 File Uploads: Multer and Local Disk vs. Cloud Object Storage

Lab reports and scans need to be uploaded, stored, and linked back to the correct patient. This project uses Multer, an Express middleware that parses multipart form data and saves the file to local disk, while a Mongoose document records the filename, MIME type, size, a type tag, and a reference to the patient. The alternative seriously considered was cloud object storage such as Amazon S3, the more common approach for a system expected to scale across many clinics, but local disk was chosen for deployment simplicity — no external account, API keys, or extra monthly cost. The trade-off is that local disk doesn't scale across multiple server

instances and makes backups the clinic's own responsibility, flagged in the Future Scope chapter as a straightforward upgrade later.

### 2.4.5 The Role of AI in Clinic Software

Large language models increasingly show up in healthcare-adjacent software — summarizing a long patient history, drafting routine administrative responses, or acting as a basic chatbot over structured data. This project integrates a cloud-based Generative AI service to provide two practical features: automatic summarization of patient medical history for doctors and an AI assistant that helps patients understand reports and medical terminology. The AI operates as a supportive tool, while all clinical decisions remain the responsibility of qualified healthcare professionals.

### 2.4.6 Frontend Approach: Plain JavaScript vs. a Framework

A related decision was whether to build the frontend with a framework such as React, or plain HTML, CSS, and JavaScript. Frameworks make managing UI state easier as an app grows, through a component model and predictable data flow, but this project uses plain JavaScript instead, for reasons similar to Section 2.4.1: the three dashboards are each relatively simple and self-contained, without deeply nested UI state that would make manual DOM updates painful. React would have added a build step and a component-based mental model not strictly necessary here. The trade-off is real, though — as more interactive features are added, plain JavaScript's lack of built-in state management would become a genuine maintenance burden, and a framework migration would likely make sense at that point.

### 2.4.7 Password Storage: bcrypt vs. Plain or Reversibly Encrypted Storage

Storing a password as plain text is an obvious non-starter, and reversible encryption is only a partial fix — every password becomes recoverable if the application's encryption key is ever compromised alongside the database. This project uses bcrypt instead, built to be slow and salted by design: it generates a random salt per password so identical passwords produce different hashes, and a configurable cost factor makes brute-forcing expensive while a single legitimate login stays imperceptible to the user. Critically, bcrypt is one-directional — there's no way back to the original password — so even a full database breach doesn't hand over usable passwords, which is exactly why it was chosen over a reversible scheme.

### 2.4.8 Deployment Model: Single VPS vs. Managed Cloud Platform

A final consideration, tied to the cost concerns throughout this chapter, is how the finished system would actually be hosted for a real clinic. Large managed cloud platforms, which automatically handle scaling and load balancing, are the standard approach for a system expected to grow unpredictably or serve many customers — broadly the model Practo runs on. For a single small clinic, though, a modestly specified virtual private server running the backend, a MongoDB instance, and the static frontend is a far more proportionate starting point: a fixed, predictable monthly cost matching the actual load of one clinic's appointment volume, nowhere near the scale that would justify an auto-scaling platform. This fits the theme running through the whole chapter — at every layer, the simpler, more directly controllable option was chosen because the actual scale this project serves doesn't yet justify the added cost of the more "enterprise" choice.

### 2.4.9 Testing Approach: Manual API Testing vs. Automated Test Suites

Large commercial platforms typically rely on automated test suites — unit, integration, and end-to-end tests — the generally recommended approach for any codebase maintained by a team over a long period, since automated tests catch regressions immediately.

For this project, API endpoints were tested manually using Postman — login, appointment creation, prescription writing, file upload — checking responses and database state by hand. Scenarios included booking two overlapping appointments to confirm the conflict is rejected, uploading an unsupported file type to confirm Multer's filter blocks it, and sending an expired or tampered JWT to confirm the route returns unauthorized. This was a conscious trade-off: for a fixed deadline and a single developer, time was judged better spent completing the feature set. The real cost is that there is no safety net catching a regression later without re-testing by hand; automated tests are listed in the Future Scope chapter as a valuable addition.

## 2.5 Gap Analysis

Section 2.3.5 already noted that none of the four reviewed approaches is a poor system on its own terms. The point of a gap analysis is to take the specific combination of constraints a small clinic actually has — cost, staffing, and a narrow three-role scope — and check it directly against what each option offers.

- Manual and spreadsheet setups fail on access control and data linkage — no role separation, nothing structurally connecting an appointment to its prescription or notes.
- Practo Ray is a strong commercial product, but it couples clinic management to a broader marketing platform and covers considerably more scope than a small, single-clinic tool needs.
- OpenMRS is free and thorough, but expects in-house technical capacity to configure, and is oriented around clinical and public-health record-keeping rather than a receptionist-managed booking loop.
- Enterprise HIS platforms solve most underlying problems, but are priced and architected for multi-department hospitals, not a two- or three-doctor practice.
- ABDM points toward interoperable digital records as the clear direction of travel, but adopting that framework directly adds regulatory and technical integration out of proportion to what a small clinic needs simply to get off paper.

What is missing across all of these, for a clinic this project's size, is a single tool simple enough to self-host without a dedicated IT team, cheap enough to have no ongoing licensing cost, and narrow enough to map cleanly onto exactly three roles without unrelated modules for billing, insurance, or marketing. Mapped onto the objectives from Section 1.4: role-based dashboards trace back to the access-control failure in manual setups; keeping the system self-hostable traces back to Practo's subscription pricing; keeping the workflow narrow traces back to OpenMRS's deployment overhead; and staying clear of billing and insurance traces back to the scope mismatch in both the Practo and enterprise HIS reviews.

## 2.6 Summary

This chapter looked at how clinic software has evolved from paper registers to today's mix of enterprise HIS, consumer products like Practo, and open-source systems like OpenMRS, including the push toward digital health infrastructure in India through ABDM, and at why each falls short for a small clinic despite being well-built within its own scope.

It then walked through this project's own technology choices — Node.js and Express over Django or Spring, MongoDB over a relational database, JWT over sessions, and local disk storage over cloud object storage — none claimed to be universally correct, only better suited to this project's scale, budget, and timeline. The gap analysis in Section 2.5 ties these threads together: simpler than an enterprise HIS, more self-contained than Practo, easier to deploy than OpenMRS, and narrower in scope than any of the three.

It's worth being honest about what this chapter doesn't claim. It does not claim the HealthCare Portal is a better system than Practo Ray or OpenMRS in any general sense — both are more capable and more widely deployed than a single vocational project could match. What it argues, more narrowly, is that for the specific constraints a small clinic actually has, a smaller, self-hostable tool fills a real gap none of the larger platforms were designed to fill. Chapter 3 restates this gap as a formal problem statement ahead of the system design in Chapter 4.

## Sources Referenced in This Chapter

The following documentation and program pages were used directly in preparing this chapter, in place of invented academic citations:

- OpenMRS — openmrs.org, and the Regenstrief Institute's OpenMRS overview (regenstrief.org)
- Practo Ray — practo.com/providers/clinics/ray, and the Practo Digest blog (blog.practo.com)
- Ayushman Bharat Digital Mission — National Health Authority program pages (abdm.gov.in, pib.gov.in)
- Node.js and Express.js official documentation (nodejs.org, expressjs.com)
- MongoDB and Mongoose official documentation (mongodb.com/docs, mongoosejs.com)
- JWT.io — introduction to JSON Web Tokens (jwt.io/introduction)

# CHAPTER 3: PROBLEM IDENTIFICATION

## 3.1 Existing Problems

Chapters 1 and 2 already went through this in some detail, so this section just pulls the main points together in one place before stating the problem formally. Small clinics that don't run any dedicated software are, in practice, stuck with one of two setups: a fully paper-based process, or a loose mix of general-purpose digital tools like spreadsheets and messaging apps standing in for a real system.

Both of these lead to the same three recurring issues. Scheduling is opaque — a receptionist working from a single register or a single spreadsheet can't easily give other staff, let alone patients, a live view of what's booked and what's free, which leads to double-bookings and unnecessary phone calls. Patient history is fragmented — a doctor seeing a returning patient usually has no fast way to check what was diagnosed or prescribed during an earlier visit unless they personally remember it or someone manually retrieves the old file. And documents get lost — lab reports and scans handed to a patient on paper are easy to misplace between visits, which sometimes means a test has to be repeated for no reason other than the original result being unavailable.

None of these three issues are dramatic on their own, which is part of why they tend to persist rather than getting fixed. A single double-booking is an inconvenience, not a crisis. A doctor missing one detail from a past visit usually doesn't change the outcome. But these things compound over time — a clinic seeing dozens of patients a day, week after week, accumulates a steady stream of small scheduling conflicts, small gaps in history, and small pieces of paperwork that go missing, and the cost shows up as wasted staff time, repeated tests, and a slightly worse experience for every patient involved, rather than as one obvious failure.

The commercial and open-source options reviewed in Chapter 2 don't really close this gap for a single small clinic either. Platforms like Practo are built around a nationwide marketplace and a subscription fee, and systems like OpenMRS are built to be endlessly configurable for

very different healthcare contexts, which comes with setup work most small clinics aren't equipped to take on. What's missing, specifically, is a small, self-contained system built around the exact workflow a small clinic needs, without the cost of one option or the configuration overhead of the other.

## 3.2 Problem Statement

Small clinics that rely on paper registers or disconnected digital tools have no reliable way to manage appointments in real time, keep a patient's prescriptions and notes linked to their visit history, or store medical documents against the correct patient record. This leads to scheduling conflicts, doctors working without a full picture of a patient's past treatment, and documents that go missing between visits. Existing alternatives either target a much larger scale than a single clinic needs (enterprise HIS platforms, nationwide booking marketplaces) or require more setup and technical support than a small clinic can reasonably provide (fully configurable, general-purpose EMR platforms).

## 3.3 Need for the Proposed Solution

Given this, there's a reasonably clear need for a lightweight, role-based system that a small clinic could adopt directly, without a subscription fee, without needing to configure a general-purpose platform from scratch, and without needing dedicated IT staff to keep it running. It needs to give the receptionist, doctor, and patient a shared, real-time view of the same appointment data, tie prescriptions and notes directly to the visit that produced them, and let patients upload their own documents so they aren't relying on paper copies surviving between visits.

It also needs to be something a small clinic can realistically start using without a long transition period. A system that requires weeks of data migration, staff retraining, or upfront configuration work isn't much of an improvement over paper if the clinic can never actually get to the point of using it day to day. This is part of why the proposed solution, described below, keeps its data model and its three dashboards deliberately narrow — the faster a receptionist, doctor, and patient can each understand their own dashboard, the more likely the system is to actually get used rather than abandoned after a few weeks.

## 3.4 Proposed Solution

The HealthCare Portal is built to be exactly this: a single web application with three role-based dashboards, backed by a database where appointments, prescriptions, notes, and uploads are all linked to the correct patient and doctor rather than existing as separate, disconnected records. A patient can book and track an appointment themselves. A receptionist can see and manage the full schedule instead of a single physical register. A doctor can pull up a patient's linked history — past prescriptions, notes, and uploaded documents — instead of relying on memory or a manually retrieved paper file.

This doesn't require the clinic to pay a recurring platform fee, and it doesn't require configuring a large, general-purpose system before it becomes useful. The scope is deliberately narrow, covering the specific workflow described above rather than every feature a hospital-scale system might eventually need, which is exactly what keeps it realistic for a small clinic to actually adopt.

## 3.5 Expected Outcome

If the system works as intended, the three recurring problems from Section 3.1 should each show a measurable improvement, even if none of them disappear completely. Scheduling conflicts should drop, since the receptionist, doctor, and patient are all looking at the same appointment status instead of separate, possibly out-of-date sources. A doctor treating a returning patient should be able to check that patient's past prescriptions and notes in seconds rather than not checking at all. And a patient's lab reports and scans, once uploaded, should stay attached to their record permanently instead of depending on a physical copy surviving between visits.

The broader outcome is less about any single metric and more about whether the clinic actually keeps using the system after the first few weeks. A tool that looks good in a demo but gets abandoned once the novelty wears off hasn't really solved anything. How well the system holds up against this — both technically and in terms of whether the workflow genuinely fits how a small clinic operates day to day — is covered in the Results and Discussion chapter, after the system's design and implementation are described in Chapter 4.

# CHAPTER 4: METHODOLOGY

This chapter goes through how the HealthCare Portal was actually designed and put together — the overall architecture, the development approach used, the technology stack, the database design, and the UML diagrams that describe how the system behaves. Where a diagram is genuinely useful for understanding the system, one is included directly in this chapter rather than described only in words.

## 4.1 System Architecture

The system follows a fairly standard three-layer Client-Server architecture, split into a client layer, a server layer, and a data layer. Keeping these layers separate was a deliberate choice — the client doesn't know anything about how MongoDB is structured, and the database doesn't know anything about how the dashboards are rendered. Each layer only needs to understand the layer immediately next to it.

The client layer is a browser-based interface built with plain HTML, CSS, and JavaScript. It doesn't do any real processing of its own beyond form handling and rendering — every meaningful action (booking an appointment, writing a prescription) results in an HTTP request to the server layer. The server layer is an Express.js application running on Node.js, organized around REST routes for each resource (appointments, prescriptions, notes, uploads, users). Two pieces of middleware sit in front of these routes: a JWT authentication middleware that checks the token on incoming requests and attaches the user's id and role to the request, and Multer, which intercepts file upload requests specifically and writes the uploaded file to disk before handing control back to the route handler. The data layer is MongoDB, accessed through Mongoose, which is where the five core collections — Users, Appointments, Prescriptions, Notes, and Uploads — actually live. The system also integrates the Gemini API for AI-powered patient summaries and the Ask AI feature. All AI requests pass through the Express.js server, ensuring secure and authenticated access. This keeps the AI service separate from the client while allowing it to integrate seamlessly with the existing healthcare workflow.

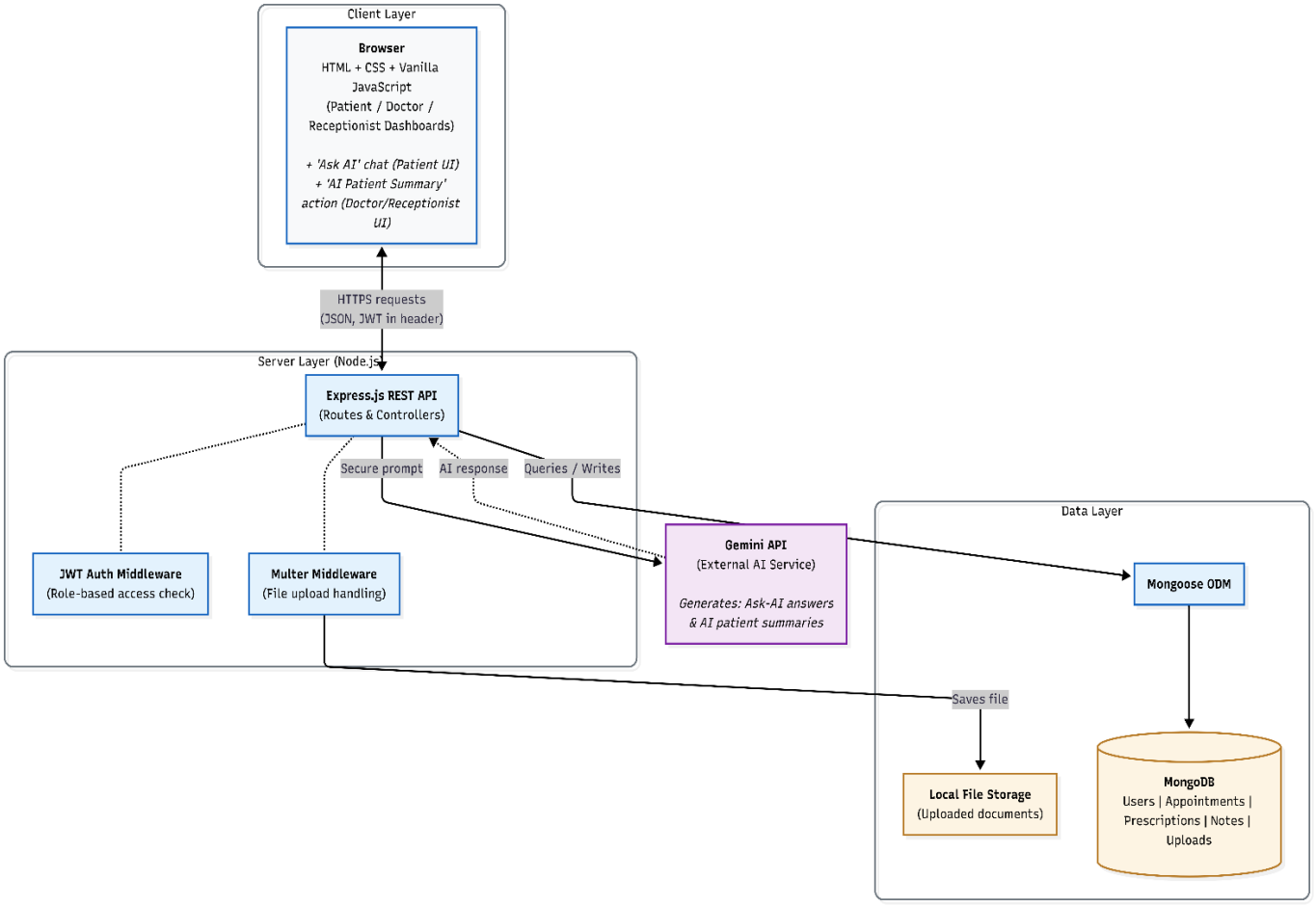


*Fig 4.1 — System architecture: client, server, and data layers.*

One thing worth pointing out about this diagram is that the browser never talks to MongoDB directly, and never could — every request has to go through the Express API, which is also where the JWT check happens. This means access control is enforced in exactly one place rather than being scattered across the client code, which would be far easier to bypass.

### 4.1.1 Why Keep the Layers This Separate

It would have been technically possible to blur these boundaries a little — for example, having the frontend construct MongoDB-style queries and send them directly to a thin proxy, rather than going through a full REST API. This was avoided specifically because it would have meant re-implementing access control on the client side, in JavaScript the browser can inspect and modify, rather than on the server, where the person calling the API has no ability to see or

change the code enforcing the rules. Every rule about who can see or modify what data lives in exactly one place: the Express route handlers and their middleware.

### 4.1.2 Request Flow for a Typical Action

It's worth walking through what actually happens, layer by layer, for a single request — say, a doctor submitting a prescription. The browser collects the form data and sends a POST request to /api/prescriptions with the JWT attached in the Authorization header. The JWT middleware on the server verifies the token's signature, confirms it hasn't expired, and reads the doctor's id and role from its payload. Only after this check passes does the request reach the actual route handler, which validates the request body against the Prescription schema, saves it through Mongoose, and returns the saved document as JSON. The browser then updates the doctor's dashboard based on that response. At no point does the browser get direct access to the database, and at no point does the database layer need to know anything about roles or permissions — that logic lives entirely in the middleware layer in between.

### 4.1.3 Scalability Notes

The architecture as built runs as a single Express process talking to a single MongoDB instance, which is appropriate for one clinic's worth of traffic but would need a few changes to scale further. Because JWT verification doesn't depend on server-side session storage, running multiple Express instances behind a load balancer would work without any change to the authentication logic, as discussed in Chapter 2. MongoDB itself would more likely need to move from a single instance to a replica set for redundancy before being trusted with production patient data at any real scale. Neither of these changes was necessary for this project, but the choice of JWT over sessions in particular means the system isn't architecturally blocked from scaling this way later, even though it wasn't built with that requirement in mind from day one.

## 4.2 Software Development Life Cycle

The project followed an Agile-influenced, iterative approach rather than a strict Waterfall model, mainly because the exact shape of the data (what fields a prescription needs, what

statuses an appointment should support) wasn't fully obvious until some of it was actually built and tested. Working in short iterations — building one collection and its routes, testing it, then moving to the next — made it easier to adjust the schema design early, before too much of the frontend depended on it.

### 4.2.1 Requirement Gathering and Planning

The starting point was identifying the three roles and working out, in plain terms, what each one needed to be able to do. This produced the feature list in Chapter 1 (booking, prescriptions, notes, uploads) before any code was written, and that list stayed fairly stable throughout development, even as the underlying schema details changed. A rough priority order was also set at this stage — authentication and appointments first, since nothing else in the system makes sense without them, followed by prescriptions and notes, with uploads treated as the last core feature to implement.

### 4.2.2 Design

With the feature list settled, the next step was sketching out the data model — deciding on the five collections and how they reference each other — followed by a rough plan for the API routes each collection would need. This is also where the choice of Node.js, Express, MongoDB, and JWT was finalized, based on the reasoning covered in Chapter 2. The database design in Section 4.4 and the ER diagram in Fig 4.2 are the direct output of this design phase, though a few fields (like feePaid) were added slightly later once the appointment workflow was better understood.

### 4.2.3 Implementation

Implementation happened collection by collection rather than building the entire backend before touching the frontend. The User model and authentication came first, since every other route depends on knowing who's making the request and what role they have. Appointments came next, followed by Prescriptions and Notes, and Uploads last, since it depended on Multer being configured correctly. The frontend was built alongside each collection rather than afterward — once the appointment routes existed and were tested, the patient and receptionist appointment screens were built against them immediately, rather than waiting for every backend route to be finished first.

### 4.2.4 Testing

Each route was tested individually using Postman as it was built, rather than waiting until the whole backend was finished to start testing. This made it much easier to catch a broken route immediately, while the code that caused it was still fresh, rather than trying to debug a large batch of untested routes all at once later. Once a collection's routes were confirmed to work in isolation, the corresponding frontend screens were tested manually against them, checking that role restrictions actually blocked the requests they were supposed to block, not just that they allowed the ones they should. Testing is covered in more detail in Chapter 5.

### 4.2.5 Deployment and Iteration

Because the project was built iteratively, minor design changes — such as adding the feePaid field to the Appointment schema partway through development — were folded in as they came up, rather than requiring a full redesign. This iterative approach is really the main reason an Agile-style process fit better than Waterfall for a project of this size and timeline. A strict Waterfall approach would have required the entire database schema to be finalized before any implementation began, which would have been difficult here, since some of the schema decisions (like exactly which fields a prescription's medicine entries needed) only became clear once the prescription form was actually being built and tested against real sample data.

## 4.3 Technology Stack

The reasoning behind each of these choices was covered in Chapter 2; this section lists the concrete stack as implemented, along with a short note on the specific role each piece plays.

### 4.3.1 Frontend

- HTML5 and CSS3 for structure and styling — no CSS framework was used, since the number of distinct screens (roughly three dashboards plus login/registration) didn't justify the overhead of setting one up.
- Vanilla JavaScript for form handling, API calls (via fetch), and DOM updates — no frontend framework, since each dashboard's interactivity is fairly limited (forms, lists, status updates) and didn't need component-level state management.

### 4.3.2 Backend

- Node.js as the JavaScript runtime, chosen so the same language could be used on both the client and server.
- Express.js for routing and middleware, kept deliberately thin rather than layered with additional abstraction on top.
- jsonwebtoken for creating and verifying JWTs, used directly rather than through a larger authentication framework.
- bcrypt for password hashing, used only at registration (to hash) and login (to compare).
- Multer for handling multipart file uploads, configured to store files under a dedicated uploads directory rather than mixed in with application code.

### 4.3.3 Database

- MongoDB as the document database, running as a single instance for this project rather than a replica set, which would be the norm for a production deployment.
- Mongoose as the ODM layer, providing schemas, validation, and population of referenced documents — for example, populating a doctor's name and specialization directly onto an appointment document when it's fetched, rather than requiring a second manual lookup.

### 4.3.4 Tools

- Postman for testing API routes during development, including saved request collections for each set of routes.
- Git for version control, with commits roughly aligned to the collection-by-collection implementation order described in Section 4.2.3.
- VS Code as the primary editor.

Everything in this stack is open source and free to run, which was a deliberate constraint rather than an accident — Chapter 1 and Chapter 2 both go into why that mattered for a project aimed at small clinics with limited budgets.

## 4.4 Database Design

The database is organized around five Mongoose collections: Users, Appointments, Prescriptions, Notes, and Uploads. Rather than a single flat structure, each collection is kept fairly narrow and linked to the others through ObjectId references, which keeps each schema easy to reason about on its own.

### 4.4.1 Users

A single Users collection holds all three roles — patient, doctor, and staff — distinguished by a role field rather than three separate collections. This was a deliberate simplification: since login and authentication work the same way regardless of role, keeping them in one collection avoided duplicating that logic three times. Fields specific to only one role, such as specialization and qualification for doctors, are simply left blank for users of other roles.

### 4.4.2 Appointments

Each Appointment references a patient and a doctor by ObjectId, along with a date, a slot, a status (restricted to pending, waiting, inProgress, done, or cancelled through a Mongoose enum), and fields for fee and feePaid to support future payment integration.

### 4.4.3 Prescriptions and Notes

Prescriptions and Notes both reference the appointment they came from, along with the doctor and patient involved. Keeping this link to the originating appointment, rather than just to the patient directly, means the system can show exactly which visit produced a given prescription or note, rather than a flat, undated list.

### 4.4.4 Uploads

Uploads reference the owning patient and store file metadata — original name, stored filename, mimetype, size, a type tag (labReport, scan, prescription, or other), and an optional report date — separately from the actual file, which is kept on disk by Multer.

### 4.4.5 Why Five Separate Collections Rather Than Embedding

MongoDB allows related data to be embedded directly inside a parent document instead of split into separate collections — for example, a patient's prescriptions could, in principle, live as an array embedded inside their User document rather than as a separate Prescriptions collection. This was deliberately avoided here. A patient's prescriptions, notes, and uploads can each grow indefinitely over years of visits, and embedding all of that inside a single User document would make that document larger and slower to load every time the system just needs basic patient details, such as when populating a dropdown of patients for a doctor to choose from. Keeping these as separate, referenced collections means a route that only needs

a patient's name and contact details doesn't have to load their entire prescription history along with it.

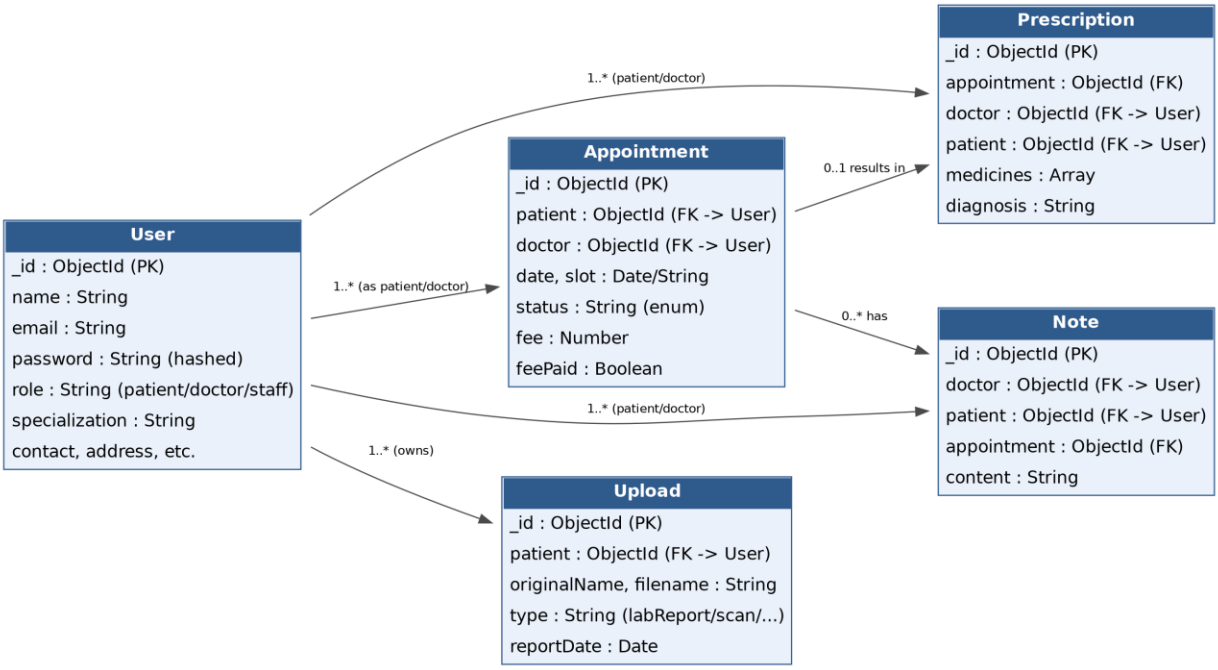


*Fig 4.2 — Entity-relationship diagram showing the five core collections and their references.*

A few relationships are worth calling out specifically. A User can be linked to many Appointments, either as the patient or the doctor on that appointment. An Appointment can lead to at most one Prescription, but can have several Notes attached to it if the doctor adds more than one. Uploads are linked only to the patient, not to a specific appointment, since a lab report might be relevant across several future visits rather than just the one during which it was uploaded.

### 4.4.6 Indexing Considerations

Beyond the schema shape itself, a couple of fields are natural candidates for indexing once the system has enough data for query performance to matter — particularly the patient and doctor reference fields on the Appointment collection, since most queries either filter appointments by a specific patient or a specific doctor. For the scale this project was tested at, MongoDB's default indexing on the _id field was sufficient, but this is one of the more obvious places a production deployment with a larger patient base would need to add explicit indexes to keep dashboard queries fast.

### 4.4.7 Schema Validation Rules

Each Mongoose schema defines which fields are required and which are optional, rather than leaving every field technically optional and relying on the frontend to enforce completeness. An Appointment, for example, cannot be saved without a patient, a doctor, a date, and a slot — Mongoose rejects the save attempt and returns a validation error before anything reaches MongoDB. The status field goes further, using an enum restriction so that only the five defined values (pending, waiting, inProgress, done, cancelled) are ever accepted; an attempt to set it to anything else fails validation rather than silently saving an invalid status that the rest of the system wouldn't know how to handle.

This matters in practice because the frontend is not the only thing that could ever talk to this API — a bug in the frontend code, or someone testing the API directly through Postman, could otherwise send a malformed request. Enforcing these rules at the schema level means the database itself refuses bad data, rather than the correctness of the whole system depending on the frontend always behaving exactly as expected.

## 4.5 UML Diagrams

Three diagrams are included here to describe the system from different angles: a use case diagram showing what each role can do, a sequence diagram showing how a single appointment booking actually flows through the system step by step, and an activity diagram showing the appointment's lifecycle as it moves between statuses.

### 4.5.1 Use Case Diagram

The use case diagram lays out the three actors — Patient, Doctor, and Receptionist — against the specific actions each one can take. It's really a visual version of the feature list from Chapter 1, but laid out so it's easy to see that none of the three roles' use cases overlap except for the shared Register/Login action.

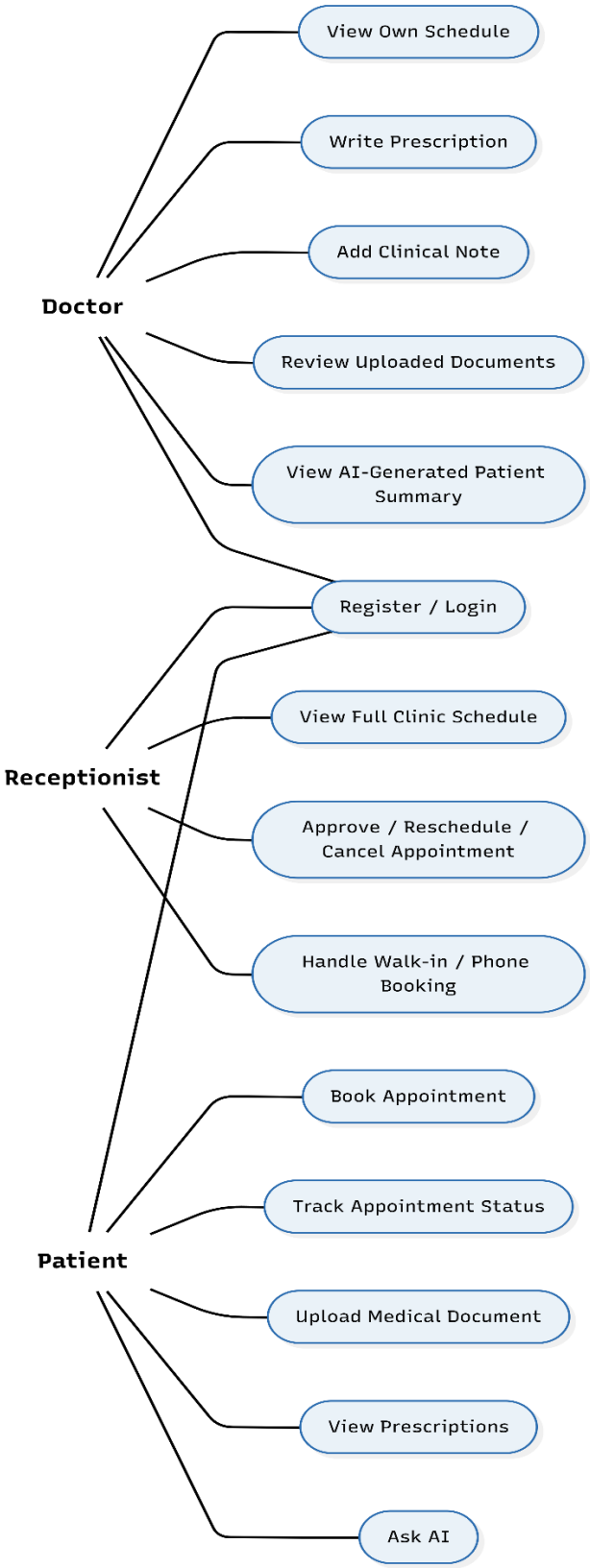


*Fig 4.3 — Use case diagram for the three system roles.*

### 4.5.2 Sequence Diagram: Booking an Appointment

The sequence diagram below traces a single booking from the moment a patient clicks "Book" through to the doctor completing the consultation. It follows the same walkthrough described in Chapter 1, Section 1.8, but shown as a proper sequence of messages between the patient, the frontend, the Express API, MongoDB, the receptionist, and the doctor.

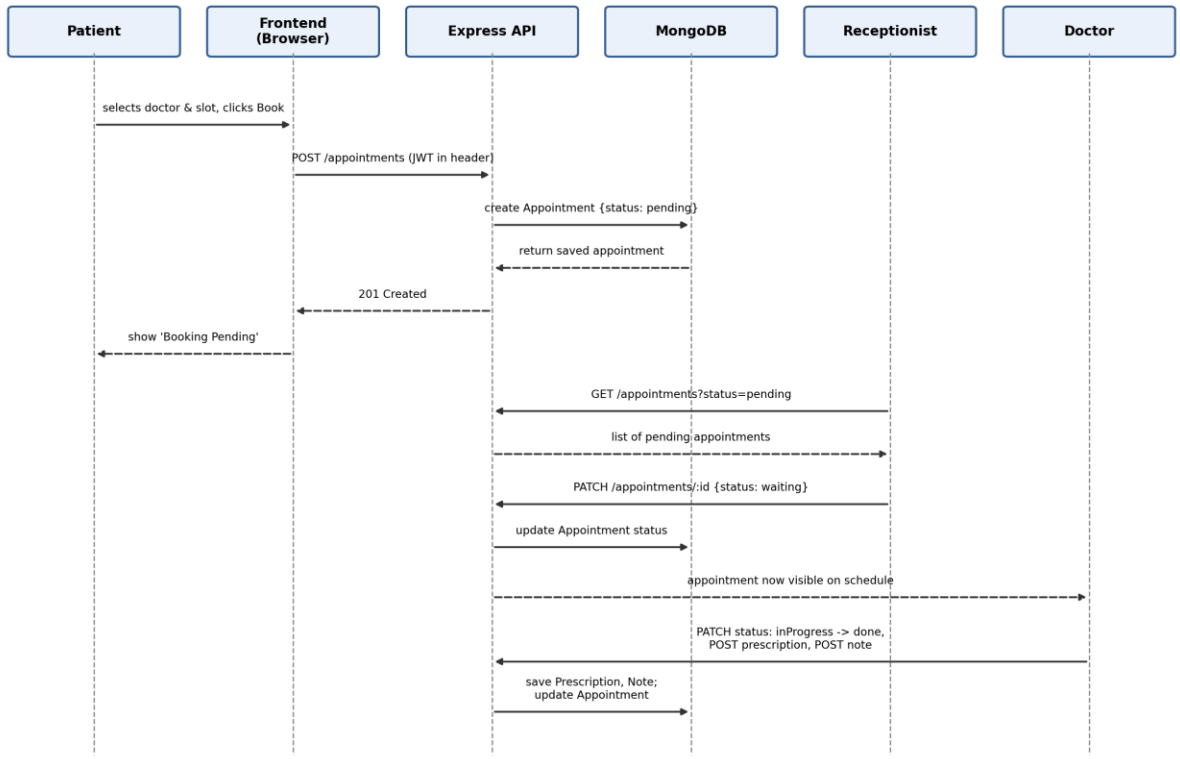


*Fig 4.4 — Sequence diagram: booking an appointment through to consultation.*

A few points from this diagram are worth calling out. Every write to MongoDB goes through the Express API — neither the receptionist's nor the doctor's dashboard talks to the database directly. And the appointment's status field is really what drives the whole diagram: each actor's next action depends on reading that status rather than on any direct communication between the patient, receptionist, and doctor.

### 4.5.3 Activity Diagram: Appointment Lifecycle

The activity diagram shows the appointment status field as a flow, from the moment it's created as pending through to done or cancelled. This is really the same lifecycle described in Chapter 1, redrawn as a flowchart rather than a narrative.

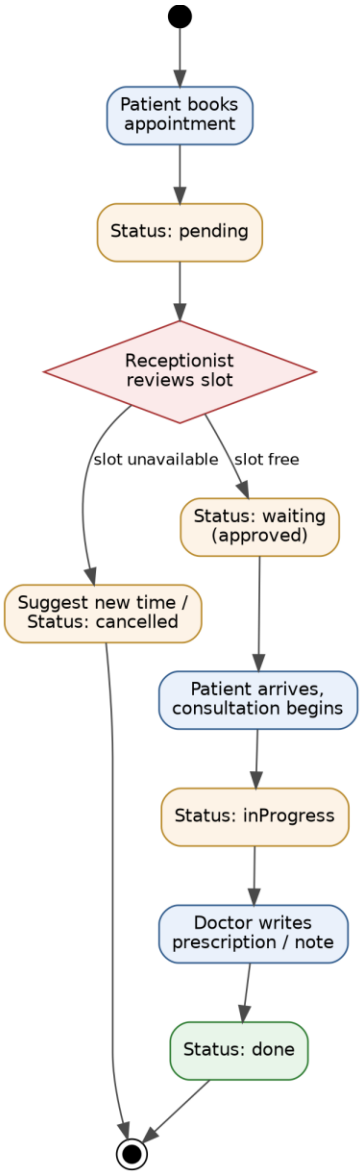


*Fig 4.5 — Activity diagram: appointment status lifecycle.*

The one branch in this flow is the receptionist's review step, where the appointment either moves forward to waiting or gets redirected — either rescheduled to a different time or cancelled outright if no slot can be found. Every other step in the diagram is linear, which

reflects how narrow the actual workflow is: there's no branching once a doctor starts a consultation, since inProgress always leads to done once the prescription and notes are recorded.

### 4.5.4 Consistency Across the Three Diagrams

It's worth noting that all three diagrams in this section are really describing the same underlying system from different angles, rather than three unrelated views. The use case diagram in Fig 4.3 lists what each actor can do; the sequence diagram in Fig 4.4 shows one specific use case (booking an appointment) played out step by step across those actors; and the activity diagram in Fig 4.5 shows the appointment's status field — the same status field referenced throughout the sequence diagram — as a standalone flow. If any of the three diagrams disagreed with the others (for example, if the sequence diagram showed a doctor approving an appointment, which is actually a receptionist-only action according to the use case diagram), that would be a sign the diagrams didn't match the actual implementation. Keeping them consistent with each other, and with the schema in Section 4.4, was treated as a basic correctness check on the design itself, not just a documentation nicety.

### 4.5.5 End-to-End Request Processing Workflow

The three diagrams above each describe one angle of the system, but it also helps to see a single request traced start to finish through every layer it actually passes through — from the user clicking something in the browser to the database being read or written and a response coming back. Fig 4.6 lays this out as a flowchart, using a generic action (login, booking, writing a prescription, or uploading a file all follow the same path) rather than any one specific case, with the two authentication-related failure branches shown explicitly since they come up constantly during testing.

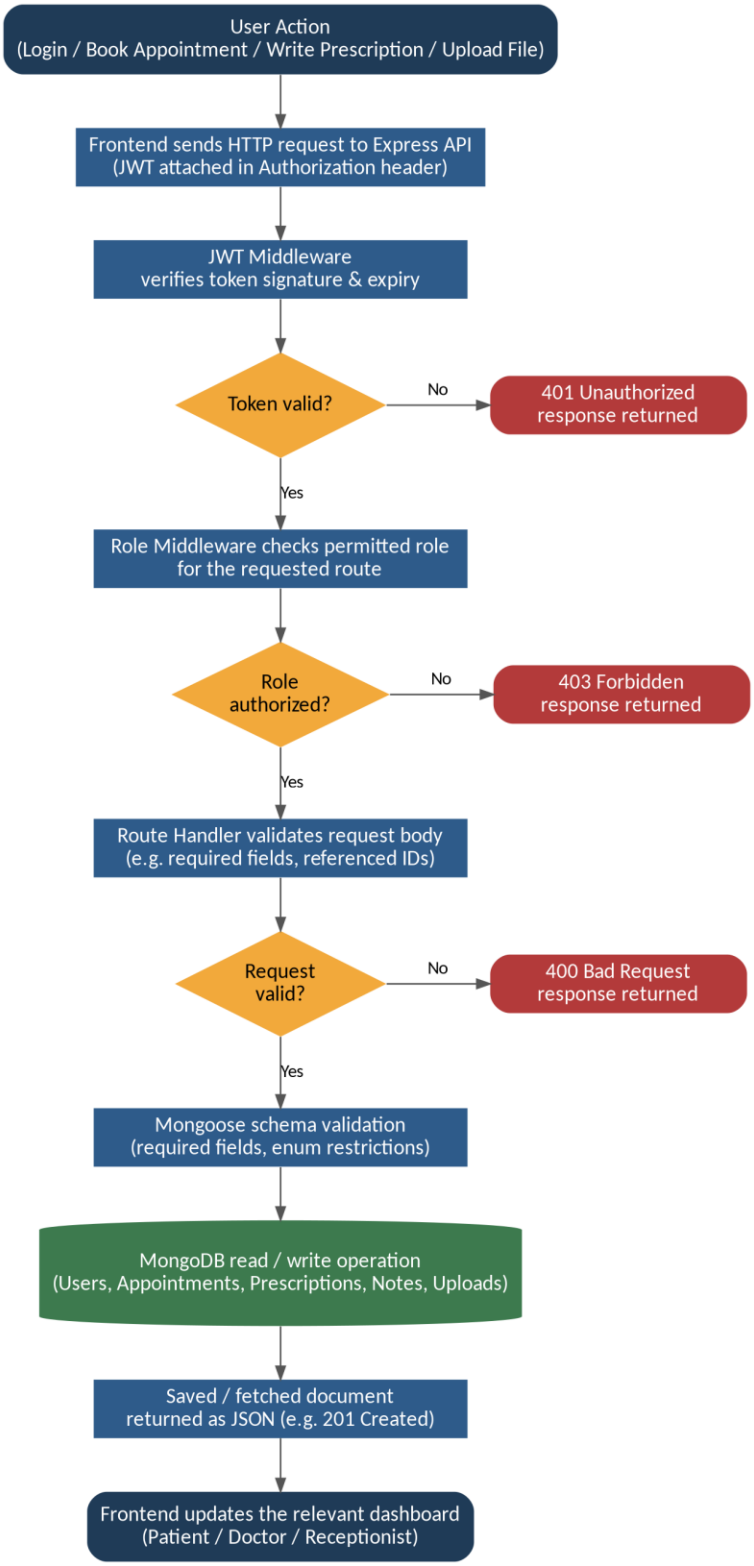


*Fig 4.6 — End-to-end request processing workflow: a single request traced from user action through authentication, validation, and the database, to the response.*

The two decision points shown — token validity and role authorization — are exactly the JWT and role-check middleware described in Section 4.1 and again in Section 4.7.1, just drawn here as explicit branches rather than described in prose. A third check, request-body validation, sits at the route-handler level rather than the middleware level, which is why it's drawn after the role check rather than alongside it: a request can be from a fully authenticated, correctly-authorized user and still fail this step if a required field is missing. Only once all three checks pass does the request actually reach Mongoose and, in turn, MongoDB.

This flowchart is deliberately generic rather than tied to one endpoint, since the same three-check pattern — authenticate, authorize, validate — repeats across every route listed in Section 4.6.1. A route that skipped one of these checks would be the exception rather than the rule, and would be treated as a bug rather than a valid shortcut.

## 4.6 API Design Overview

The backend exposes a REST API, with each collection getting its own base route and the usual HTTP verbs mapped onto create, read, update, and (where relevant) delete operations. Every route except registration and login requires a valid JWT, and most routes additionally check the role attached to that token before allowing the request through — for example, only a user with the doctor role can hit the route that creates a prescription.

### 4.6.1 Representative Routes

| HTTP Method | API Endpoint | Permitted Roles | Request Payload / Description | Primary Status Codes |
|---|---|---|---|---|
| POST | /api/auth/register | Public / Unauthenticated | Account registration details (hashes password via bcrypt) | 201 Created, 400 Bad Request |
| POST | /api/auth/login | Public / Unauthenticated | Email and password credentials; returns JWT token on success | 200 OK, 401 Unauthorized |
| GET | /api/patient/ appointments (and doctor/reception equivalents) | Patient, Doctor, Receptionist | Fetches filtered appointments scoped to the user's role | 200 OK, 401 Unauthorized |
| POST | /api/patient/ appointments/book | Patient | Doctor ID, requested date, and time slot | 201 Created, 400 Bad Request |
| PATCH | /api/reception/ appointments/:id/ status | Doctor, Receptionist | Updates status (pending, waiting, inProgress, done, cancelled) | 200 OK, 403 Forbidden |

| | | | | |
|---|---|---|---|---|
| POST | /api/doctor/ appointments/:id/ prescription | Doctor | Appointment ID, diagnosis, and array of medicines (dosage, duration) | 201 Created, 403 Forbidden |
| POST | /api/doctor/ appointments/:id/ notes | Doctor | Appointment ID, Patient ID, and clinical note string | 201 Created, 403 Forbidden |
| POST | /api/patient/uploa d | Patient | Multipart form data processed via Multer (lab report, scan, or prescription) | 201 Created, 400 Bad Request |

*Table 4.1 — REST API Endpoint Architecture and Role Access*

### 4.6.2 Why This Shape

Keeping each collection's routes grouped under its own base path (/api/appointments, /api/prescriptions, and so on) rather than a single combined endpoint made it easier to apply role checks consistently — the middleware for a given route only needs to know which roles are allowed to hit that specific path, rather than branching on the type of data being requested inside a single, larger handler.

### 4.6.3 Error Handling and Status Codes

Routes return standard HTTP status codes rather than always returning 200 with an error message buried in the response body — a successful creation returns 201, a validation failure returns 400 with a message describing what was wrong, an unauthenticated request returns 401, and a request from a correctly authenticated but wrong-role user (a patient trying to hit a doctor-only route, for example) returns 403. This makes it easier for the frontend to handle failures consistently — a 401 anywhere in the app can trigger a redirect back to the login screen, without needing route-specific handling for that case.

### 4.6.4 Request Validation

Beyond the schema-level validation described in Section 4.4, route handlers also check a small number of request-specific conditions before writing to the database — for example, confirming that the doctor referenced on a new appointment actually has the doctor role, rather than trusting whatever ObjectId the frontend sends. This is a deliberately small set of checks rather than an exhaustive validation layer, since the goal was to catch the failure modes that

were actually observed during testing rather than defending against every conceivable malformed request.

## 4.7 Non-Functional Considerations

Beyond the functional workflow described so far, a few non-functional aspects of the design are worth calling out separately, since they don't map onto any single collection or route but affect the system as a whole.

### 4.7.1 Security

Security in this system rests mainly on three mechanisms working together: bcrypt-hashed passwords so that a database leak doesn't expose usable credentials, JWTs so that authentication doesn't depend on server-side session storage, and role checks in middleware so that a valid token for one role can't be used to access another role's routes. This is a reasonable baseline for a project at this scale, but as noted in Chapter 1, it has not gone through a formal security audit, and a production deployment handling real patient data would need additional measures — rate limiting on the login route to slow brute-force attempts, HTTPS enforced at the infrastructure level, and likely a shorter JWT expiry time paired with a refresh token mechanism.

### 4.7.2 Performance

At the scale this project was built and tested for — a single clinic with a handful of doctors and a modest, growing patient base — performance was not a major design constraint. Queries are simple, mostly filtering a single collection by one or two reference fields, and MongoDB handles this comfortably without additional tuning. Section 4.4.6 already notes where indexing would become relevant if the patient base grew substantially larger.

### 4.7.3 Maintainability

Keeping routes, models, and middleware organized by collection (a routes file, a model file, and where needed a controller file per collection) was a deliberate choice to keep the codebase navigable for a single developer or a small team. Anyone needing to modify how prescriptions work only needs to look inside the prescription-related files, rather than searching through a single large file handling every collection at once.

### 4.7.4 Portability

Because the entire stack — Node.js, Express, MongoDB — runs identically on Windows, macOS, and Linux, and because none of it depends on a paid cloud service to function locally, the system can be set up on a fresh machine with just Node.js and MongoDB installed, without any cloud account or API key. This matters specifically for the target audience described in Chapter 1: a small clinic without dedicated IT support benefits from a system that's simple to install locally before ever needing to think about hosting it somewhere for real use.

### 4.7.5 Known Design Limitations

A few limitations in the current design are worth naming plainly, on top of the feature-level exclusions already discussed in Chapter 1. The system does not currently implement soft deletes — removing an appointment or a user record deletes it outright rather than marking it as archived, which means there's no built-in way to recover a record deleted by mistake. There's also no audit trail recording who changed an appointment's status or when a prescription was edited, which a regulated clinical system would typically require. Neither of these was treated as a blocker for a project at this scale, but both are the kind of gap that would need to be closed before the system could be trusted with real, ongoing clinical use rather than a controlled demonstration.

## 4.8 Summary

This chapter went through the system's architecture, the development approach used, the technology stack, the database design, three UML diagrams describing the system's use cases, message flow, and appointment lifecycle, the shape of the REST API, and a set of non-functional considerations — security, performance, maintainability, portability, and known limitations — that don't map onto any single feature but affect the system as a whole.

Together, these describe how the objectives and problem statement from Chapters 1 through 3 were actually turned into a working system, and where the honest gaps in that system currently sit. Chapter 5 picks up from here, looking at the implementation in more detail — including actual screenshots of the running application — along with the results of testing the system once it was built.

# CHAPTER 5: RESULTS AND DISCUSSION

This chapter looks at what actually happened once the HealthCare Portal was built and run, rather than what was planned for it. How the system was built is already covered in Chapter 4; this chapter is concerned with what came out of it — the interface each role actually sees, how the system held up under testing, and an honest discussion of how the results line up against the objectives set out in Chapter 1 and the problem statement from Chapter 3.

## 5.1 System in Operation

By the time testing began, all five collections described in Section 4.4 — Users, Appointments, Prescriptions, Notes, and Uploads — were wired into working routes, and all three role-based dashboards were reachable end to end from login through to the actions specific to that role. The appointment status lifecycle walked through in Section 1.8 was the first thing tested as a full loop rather than piece by piece, since it's the one flow that touches every role: a patient books, a receptionist approves, a doctor consults and prescribes, and the patient checks the result. That full loop working correctly on the first complete run-through, without needing a patch partway through, was a reasonable sign that the pieces built separately in Chapter 4 had actually been designed to fit together rather than just individually correct.

## 5.2 User Interface Walkthrough

This section describes each screen as it actually behaves, role by role. Screenshots from the running application belong here and should be inserted directly into this section before the report is submitted — one figure per subsection below is the natural place for each.

### 5.2.1 Login and Registration

The login screen is the one page all three roles share. There's no separate URL or visual branding per role — the same form authenticates a patient, a doctor, or a receptionist, and the JWT returned after a successful login carries the role that decides which dashboard loads next. A new patient can also register from this screen; doctor and receptionist accounts are created directly by whoever administers the clinic's deployment rather than through open self-

registration, since those roles carry more access than a typical signup flow should hand out freely.

### 5.2.2 Patient Dashboard

Once logged in, a patient sees their upcoming appointments, a button to book a new one, and a way to check on past prescriptions and notes. Booking walks through picking a doctor, a date, and an open slot, and the appointment's status — pending, waiting, in progress, done, or cancelled — updates on this same screen as the receptionist and doctor act on it, so the patient never has to call the clinic just to check where things stand. Nothing about doctor schedules, other patients, or clinic-wide administration appears anywhere in this view, in line with the narrow-dashboard approach described in Chapter 1.

### 5.2.3 Doctor Dashboard

A doctor's dashboard is built around a single day's list of appointments assigned to them specifically — not the whole clinic's schedule. Opening an appointment from this list is how a doctor moves it to in progress, writes a prescription, and adds a clinical note, all from the same screen rather than jumping between separate pages for each action. Any documents the patient has uploaded ahead of the visit are also visible from here, so a doctor can review a lab report before or during the consultation rather than asking the patient to describe it.

### 5.2.4 Receptionist Dashboard

The receptionist's view is the only one that spans every doctor at the clinic at once, since approving, rescheduling, and cancelling bookings across the whole practice is specifically their job. This is also the screen where a walk-in or phone booking gets entered directly, on behalf of a patient who isn't using the app themselves — the receptionist picks the doctor and slot the same way a patient would, just from the other side of the desk.

### 5.2.5 Prescription Form

The prescription form lets a doctor add more than one medicine to a single prescription, each with its own dosage, frequency, and duration, plus a diagnosis field and free-text notes for anything that doesn't fit the structured fields. This maps directly onto the Prescription schema

from Section 4.4.3 — the medicines array on the form is exactly the medicines array saved to the document, with no translation step in between.

## 5.3 Testing and Validation

As noted in Chapter 2, this project relied on manual testing through Postman rather than an automated test suite, given the timeline and the fact that a single developer was building and testing at the same time. This section walks through what was actually tested and what came out of it, rather than presenting a polished set of numbers that would overstate how rigorous the process actually was.

### 5.3.1 API Testing with Postman

Every route was exercised individually as it was built, using a saved Postman collection organized by resource — one folder for auth, one for appointments, one each for prescriptions, notes, and uploads. Each folder included at least one request that should succeed and at least one that should fail for a specific, checkable reason, such as a missing required field or a token for the wrong role. Watching the failing requests return the right status code mattered as much as watching the successful ones return the right data — a route that only ever returns 200 regardless of what's sent isn't actually enforcing anything.

### 5.3.2 Role-Based Access Testing

Because role checks sit at the center of the system's security model, this was tested more deliberately than most other behaviour. Three separate accounts — one per role — were logged in during the same testing session, and each one's token was used to call routes meant for the other two roles. A patient's token against a doctor-only prescription route correctly returned 403, as did a doctor's token against the receptionist's cross-doctor appointment list. This is a small, manual version of the kind of test a proper test suite would run automatically, but it covered the specific failure mode that mattered most: a valid, real login being used to reach data it shouldn't be able to.

### 5.3.3 Edge Cases

A few edge cases were tested specifically because they matched problems named earlier in the report, rather than because they were the first ones that came to mind:

- Booking the same doctor, date, and slot twice in a row — the second request correctly returned a conflict error rather than creating a duplicate booking.
- Uploading a file just under and just over the configured size limit — the oversized file was rejected before it was ever written to disk.
- Uploading a file with an unsupported extension, such as a document renamed to look like an image — rejected by the upload filter rather than silently accepted and stored.
- Submitting a prescription against an appointment assigned to a different doctor, using a valid doctor token that just wasn't the right doctor — correctly rejected, since the route checks that the requesting doctor matches the one on the appointment.
- Cancelling an appointment and immediately re-booking the same slot — the new booking succeeded, confirming that a cancelled appointment doesn't permanently block a slot the way an active one does.
- Sending a malformed or missing date on a booking request — rejected with a validation error at the schema level rather than being saved with a broken or empty date field.

None of these were dramatic failures to fix — they mostly confirmed that the checks described in Chapter 4 behaved the way they were designed to. The one genuine bug caught this way was an early version of the appointment-conflict check that only looked for a status of pending, which meant a slot already moved to waiting could still be double-booked by a second request. Widening the check to also cover waiting and in-progress appointments closed that gap.

| Test ID | Test Scenario / Edge Case | Input / Action Tested | Expected System Behavior | Test Outcome |
|---|---|---|---|---|
| TC-01 | Double-Booking Conflict | Two consecutive booking requests for the exact same doctor, date, and time slot. | Rejects second booking attempt with a conflict error. | PASS |
| TC-02 | File Size Enforcement | Uploaded a document exceeding configured size limit via Multer. | Blocks upload prior to writing file to server disk. | PASS |
| TC-03 | Unsupported Extension | Uploaded non-medical/unsupported file type masked as an image. | File filter rejects upload before storage. | PASS |

| | | | | |
|---|---|---|---|---|
| TC-04 | Unauthorized Doctor Access | Submitted prescription using Doctor token against another doctor's assigned appointment. | Denies request due to assigned doctor ID mismatch. | PASS |
| TC-05 | Slot Re-booking Lifecycle | Cancelled an existing booking and immediately attempted to re-book the freed slot. | Successfully creates new appointment in released slot. | PASS |
| TC-06 | Schema Validation | Sent booking request with missing/malformed date payload. | Mongoose schema validation blocks database write. | PASS |
| TC-07 | Cross-Role Authorization | Patient token submitted against Doctor/Receptionist REST endpoints. | Middleware intercepts request and returns 403 Forbidden. | PASS |

*Table 5.1 — System Validation and Edge Case Test Results*

### 5.3.4 Manual Frontend Testing

Once a collection's routes were confirmed to work correctly on their own, the corresponding frontend screens were tested by hand — clicking through the actual booking flow, prescription form, and upload screen as a user would, rather than only testing the API in isolation. This caught a handful of smaller issues that Postman testing wouldn't have surfaced on its own, such as a status label on the patient dashboard that didn't update until the page was manually refreshed, and a file upload form that didn't show any feedback while a large file was still uploading, which made it look like nothing had happened. Both were fixed during this phase rather than left for a later iteration, since they directly affected whether the interface felt like it was working correctly, even though the underlying API calls were already correct.

### 5.3.5 What Testing Didn't Cover

It's worth being direct about the limits of this testing process rather than implying it was more complete than it was. Nothing here was load-tested — every scenario above was run by one person, one request at a time, which says nothing about how the system behaves with several doctors and receptionists hitting it simultaneously during a busy morning. There was also no test for what happens if the database connection drops mid-request, or for concurrent edits to

the same appointment from two browser tabs at once. These weren't tested not because they don't matter, but because they fall outside what manual, one-developer testing on a fixed timeline can realistically cover — the same trade-off named plainly in Section 2.4.9.

## 5.4 Objectives Revisited

Section 1.4 listed seven concrete objectives for the project. It's worth going through them individually rather than only in general terms, since a project can feel successful overall while still falling short on a specific point.

- Build a web platform connecting Patients, Doctors and Receptionists, with each role getting its own page — met. All three dashboards exist and are reachable end to end, as walked through in Section 5.2.
- Implement login and role-based access using JWT so a Patient cannot access Doctor-only or Receptionist-only data — met, and specifically verified rather than assumed, per the cross-role testing in Section 5.3.2.
- Let Patients book, view and track their own appointments without needing to call the clinic — met. The full status lifecycle from Section 1.8 is visible to the patient at every stage.
- Allow Doctors to record diagnoses, write prescriptions, and add notes tied to a Patient and appointment — met, and confirmed structurally sound by the schema-level validation discussed below.
- Give Receptionists a working view of the day's appointments so they can approve, reschedule, or cancel bookings — met.
- Support secure upload of documents linked to the correct Patient — partially met. Uploads are correctly linked and type-restricted, but "secure" is doing some work in this objective that the current system doesn't fully back up — there's no audit trail on who accessed an uploaded file, which Section 4.7.5 already named as a known gap rather than something newly discovered here.
- Reduce how much of the clinic's day-to-day work still depends on paper and manual coordination — partially met, and honestly the hardest of the seven to verify from testing alone. The system removes the need for a phone call to check appointment status and the need for a paper prescription slip, but whether a clinic's actual day-to-day habits shift away from paper is a question about adoption, not something a testing chapter can answer on its own.

The Generative AI objective described in Section 1.4 has been implemented using a cloud-based AI API. The system generates concise summaries of patient history by analysing existing prescriptions, consultation notes, and uploaded medical documents. It also provides an AI

assistant that responds to patient queries and explains medical information in simple, understandable language. Since AI-generated responses may occasionally be inaccurate, they are intended only as supportive information and not as a substitute for professional medical advice.

## 5.5 Performance and Usability Observations

Section 4.7.2 was upfront that performance was never treated as a design constraint at the scale this project targets, and nothing in this chapter contradicts that. With a modest test dataset — a handful of doctor accounts and a few dozen appointments spread across them — every screen loaded and every query returned without any noticeable delay, which is roughly what Section 4.7.2 predicted given that most queries just filter one collection by a patient or doctor reference. This is a qualitative impression from manual testing, not a benchmark, and it isn't meant to stand in for one; the indexing considerations noted in Section 4.4.6 remain something to act on once real usage numbers exist, not something this project needed to solve at its current scale.

On usability, the clearest signal came from the two frontend bugs described in Section 5.3.4 — both were the kind of thing that only shows up when someone actually uses the screen rather than just calling the API behind it. That's a small but real point in favor of the manual frontend testing pass described there: neither issue would have been caught by API testing alone, since the underlying requests were already correct in both cases.

## 5.6 Discussion of Results

Chapter 1 set out three problems this project was meant to address: scheduling conflicts, fragmented patient history, and lost documents. It's worth checking each one against what the testing in this chapter actually showed, rather than assuming success just because the features exist.

On scheduling, the conflict check confirmed in Section 5.3.3 does what it was meant to do — a slot cannot end up double-booked through the app, and this was verified directly during edge-case testing rather than just assumed from reading the code. This doesn't eliminate every scheduling problem a clinic might have, such as a doctor running late and pushing every later

appointment back, but it does close the specific gap named in Chapter 3: two people being booked into the same slot by mistake.

On patient history, linking prescriptions and notes to the appointment they came from, rather than floating loosely against just the patient, means a doctor opening a returning patient's record sees a chronological, visit-by-visit history rather than an unordered pile of notes. This matches the common scenario walked through in Section 1.3.1 — the earlier prescription and dosage are visible immediately rather than needing to be reconstructed from what the patient remembers.

On document loss, the upload feature solves the narrower problem of a file being physically misplaced after being handed to the patient, since anything uploaded through the app stays attached to the patient's record permanently. It does not solve the version of the problem where the file never reaches the system in the first place, such as a patient forgetting to upload a report at all — that remains a manual step, as noted honestly in Section 1.10.

Where the results are weaker is exactly where Chapter 4 already flagged known gaps. There is no audit trail, so a status change or an edited prescription can't currently be traced back to who made it or when — something that came up during testing as a limitation that would matter far more in an actual multi-doctor clinic than it did in the single-developer testing setup used here. Testing was also entirely manual — thorough within the specific scenarios covered in Section 5.3, but not a substitute for the kind of regression safety net an automated suite would provide once the system grows past what one person can retest by hand before every change, and Section 5.3.5 is honest about the scenarios that weren't tested at all.

A smaller but real result worth naming is how much the schema-level validation described in Section 4.4.7 ended up doing quietly. Several of the edge cases in Section 5.3.3 — the malformed date, the missing required field — never reached application-level error handling at all, because Mongoose rejected them before a save was attempted. That wasn't the original motivation for putting validation at the schema level rather than only in the route handlers, but it turned out to be one of the more useful side effects: a whole category of bad input was ruled out for free, just by having the database enforce its own shape.

Taken together, the system does what Chapter 1 set out for it to do, within the scope Chapter 1 also deliberately narrowed. It is not a finished clinical product, and it was never meant to be one at this stage — it's a working demonstration that the three core problems named in Chapter 3 can be addressed with a small, self-hostable system, without needing the cost or complexity of the larger platforms reviewed in Chapter 2.

## 5.7 Summary

This chapter walked through the system as it actually runs — the interface each role sees, the testing done against both the API and the frontend, and an honest discussion of how the results measure up against the objectives and problems set out earlier in the report. The booking-conflict and role-restriction checks held up under direct testing, the interface works end to end for all three roles, and the gaps that remain are the same ones already named in Chapter 4 rather than new surprises. Chapter 6 picks up from here, looking at what was deliberately left out of this version and what a reasonable next iteration of the system would add.

# CHAPTER 6: FUTURE SCOPE

Chapter 1 was explicit about what was left out of this version on purpose — payments, account recovery, notifications, and a dedicated mobile app — rather than treating those as oversights. This chapter goes into more detail on each one, and on what would actually be involved in adding it, since "future scope" is more useful when it's specific about the next step rather than just a wishlist.

## 6.1 Payment Gateway Integration

The Appointment schema already has a fee amount and a feePaid boolean, added early so this wouldn't need a schema migration later. What's missing is the actual payment flow: integrating a gateway such as Razorpay or Stripe, generating a payment link when an appointment is approved, and updating feePaid through a webhook once payment clears, rather than trusting the frontend to report success on its own. That webhook matters — a status only ever set by the browser after checkout can be spoofed, so the gateway needs to confirm payment server-side.

## 6.2 Password Reset and Account Recovery

This is a small addition next to the rest of this chapter, but it's a genuine gap — there is currently no way for a user who forgets their password to get back into their account except by having someone manually reset it in the database, which isn't a reasonable support path for a receptionist locked out on a busy morning. The standard fix doesn't require rethinking anything already built: a "forgot password" request generates a short-lived, single-use token stored against the user's document, emailed as a reset link, and the reset route lets the user set a new password through the same bcrypt path already used at registration. The only new dependency is a transactional email service.

## 6.3 Mobile Application

The system is currently web-only, accessed through a browser on any device, which already covers a phone browser reasonably well. A dedicated mobile app would mainly buy push notifications for appointment updates and a slightly smoother offline-tolerant experience, rather than unlocking functionality that isn't already reachable through the browser. Given that the backend is already a REST API consumed by a plain JavaScript frontend, a React Native or Flutter app could reuse the same API directly without backend changes — this is one of the more contained additions on this list.

## 6.4 Email Notifications

Right now, a patient finds out an appointment was approved, rescheduled, or cancelled only by opening the app and checking. Automatic email reminders would close that gap, hooking into the same appointment-status change that already exists in the code rather than requiring new application logic — a status update triggers a short email, sent through a transactional email service, without needing to touch the underlying booking logic at all.

## 6.5 Deletion, Archiving, and an Audit Trail

Section 4.7.5 named this as a known gap rather than a nice-to-have: the system currently deletes records outright instead of archiving them, and there's no record of who changed an appointment's status or edited a prescription. Both would need to be addressed before the system could be trusted with ongoing clinical use rather than treated as a project-scale prototype. A soft-delete flag and a small audit-log collection recording who changed what, and when, are well-understood patterns — the reason they weren't built here was time and scope, not difficulty.

None of these five items were left out because they weren't considered. Each one was cut specifically to keep this version of the project finishable within the available time, while leaving the underlying data model — the fee fields, the REST API shape, the status-based appointment lifecycle — in a state where each addition is a genuine extension rather than a rework.

# CHAPTER 7: CONCLUSION

The HealthCare Portal set out to give a small clinic something better than paper registers and disconnected spreadsheets, without the cost or complexity of a full hospital information system. Chapter 3 named three specific problems — scheduling conflicts, fragmented patient history, and lost documents — and the system built across Chapters 4 and 5 addresses each of them directly rather than as a side effect of a more general feature set.

Measured against the objectives set out in Section 1.4, the core of the project works. Three role-based dashboards exist and enforce access control through JWT and middleware-level role checks, confirmed under direct testing in Section 5.3.2 rather than only assumed from the code. Patients can book and track appointments without calling the clinic. Doctors can record diagnoses, write prescriptions, and add notes tied to a specific appointment and patient. Receptionists have a working view of the day's schedule across every doctor at the clinic. Documents can be uploaded and stay linked to the right patient permanently, addressing the most concrete version of the lost-document problem named in Chapter 3.

The Generative AI module was successfully integrated into the system to complement the core healthcare management workflow. Rather than replacing professional medical judgment, the AI provides patient history summarization for doctors and simplified medical explanations for patients. These features improve efficiency and accessibility while ensuring that all clinical decisions remain under the supervision of qualified healthcare professionals.

The system also has real limitations, and Chapters 4 and 5 were written to name them plainly rather than bury them in a features list. There is no audit trail and no soft-delete, so a mistaken edit or deletion currently has no recovery path. Testing was manual throughout, which was enough to catch the specific bugs described in Section 5.3.3 but is not a substitute for the kind of regression safety net an automated suite provides. The system has not been through a security audit, and it has not been tested at the scale a busy, multi-doctor clinic would put it under. None of these are hidden — they're the honest boundary of what a single-developer

project on a fixed timeline could reasonably cover, and Chapter 6 lays out what closing each of them would actually involve.

What this project demonstrates is narrower than a general claim that the HealthCare Portal is a better system than the alternatives reviewed in Chapter 2. It shows that a small, self-hostable tool — built with a deliberately unglamorous stack of Node.js, Express, MongoDB, and a handful of well-documented libraries — can genuinely close the specific gap a small clinic has, between an unstructured mix of paper and spreadsheets on one side and an expensive, over-built hospital system on the other. That gap was real, it was specific, and the system built here addresses it directly rather than in general terms.

On a personal level, this project was also where the reasoning behind a lot of standard web development practice stopped being something to memorize and started being something with a visible reason. Why a password is hashed and not just encrypted, why access control belongs on the server rather than the client, why a schema validates rather than trusting the frontend — these stopped being rules to follow and became decisions with a specific failure mode attached if they were skipped. That, as much as the working application itself, was the actual outcome of building this project.

# REFERENCES & BIBLIOGRAPHY

The following sources were used directly in preparing this report — official product and program documentation, and package registry pages for the libraries used in the HealthCare Portal itself — rather than invented academic citations. Each entry reflects a source that was actually consulted.

## Platforms and Programs Reviewed


1. Practo Ray — clinic practice management platform. https://practo.com/providers/clinics/ray
2. Practo Digest — company blog, referenced for platform feature descriptions. https://blog.practo.com
3. OpenMRS — open-source electronic medical record platform. https://openmrs.org
4. Regenstrief Institute — OpenMRS origin and health informatics background. https://www.regenstrief.org
5. Ayushman Bharat Digital Mission (ABDM) — National Health Authority, Government of India. https://abdm.gov.in
6. Press Information Bureau, Government of India — ABDM program announcements. https://pib.gov.in